Lecture 4. Turing Machines and Halting Problem

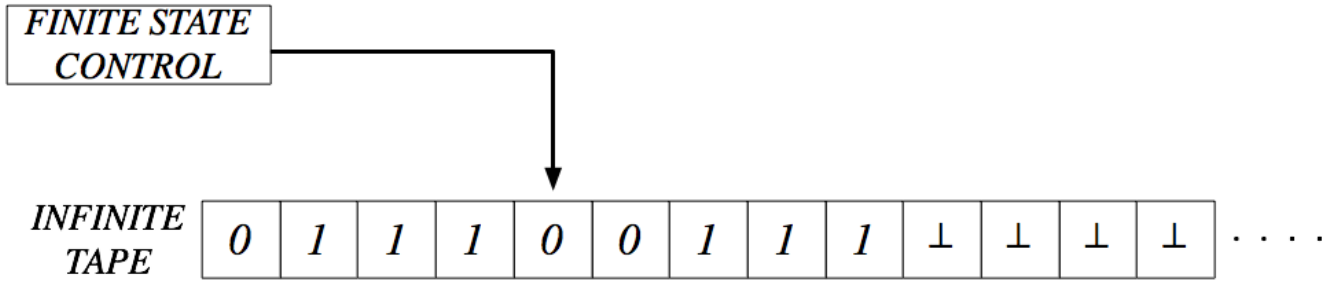
Spring 2026, *Theory of Computation*, IIIS, Tsinghua

- ***Regular languages***: (DFAs / NFAs / REGEXPs)
  - Good for pattern matching
  - Not very good at counting, e.g. $\{0^n 1^n \mid n \geq 0\}$ is irregular
- ***Context-free languages***: (Pushdown Automata)
  - Good for defining syntax/grammars (English, C++, HTML, etc)
  - Can do simple counting, e.g. $\{0^n 1^n \mid n \geq 0\}$ is context free but $\{0^n 1^n 2^n \mid n \geq 0\}$ is not
- ***Turing Machines:***
  - Can recognize ***nearly*** every language you can think of
  - …still some it can't recognize
  - *Is some model "more powerful" than Turing Machines?*



Turing Machines

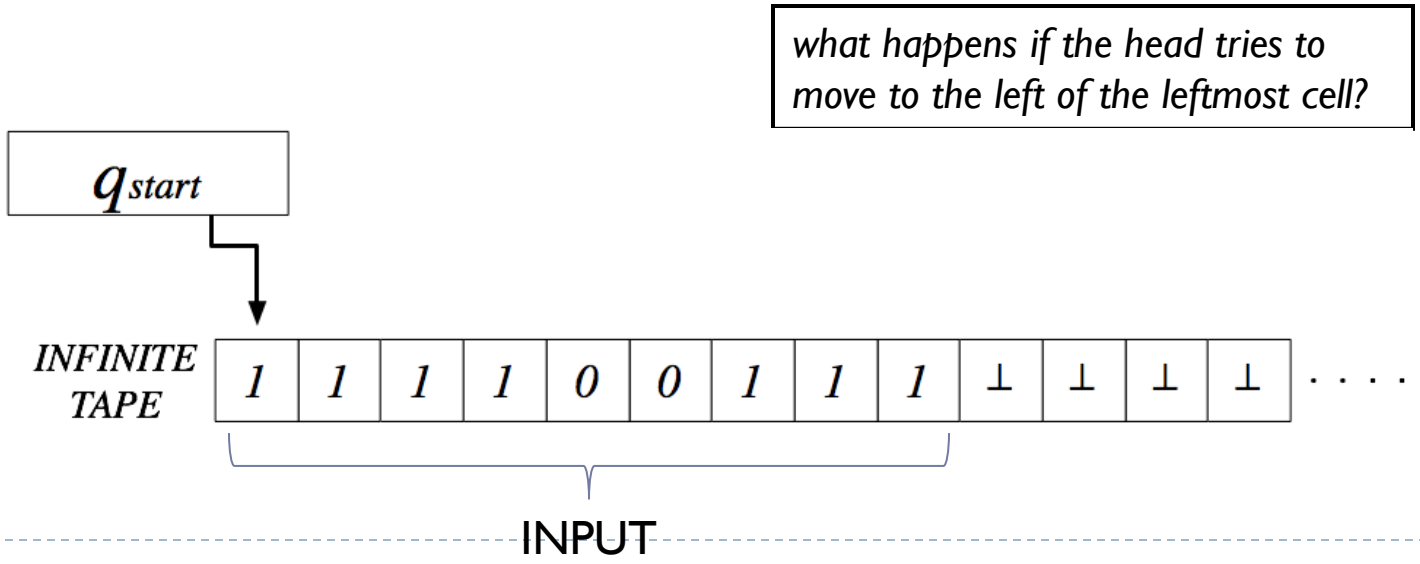
- Give a finite automaton an infinite memory:
  - The memory is an ***infinite tape*** divided into cells
  - One symbol written in each cell (or left blank: ⊥)
  - The automaton has a ***read/write*** "head" at some location on the tape
  - The head can move ***left or right***





Turing Machines

- ***In the beginning…***
  - The input ***w*** is written on the first ***|w|*** cells
  - All other cells are ⊥
  - The machine is in a special ***start state***
  - The head is at the ***leftmost cell***

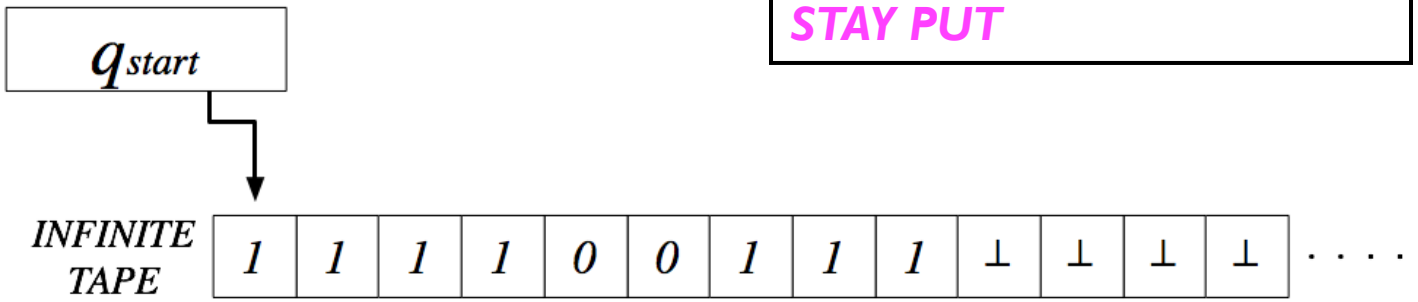




Turing Machines

- ***In the beginning…***
  - The input ***w*** is written on the first ***|w|*** cells
  - All other cells are ⊥
  - The machine is in a special ***start state***
  - The head is at the ***leftmost cell***

what happens if the head tries to move to the left of the leftmost cell?
STAY PUT



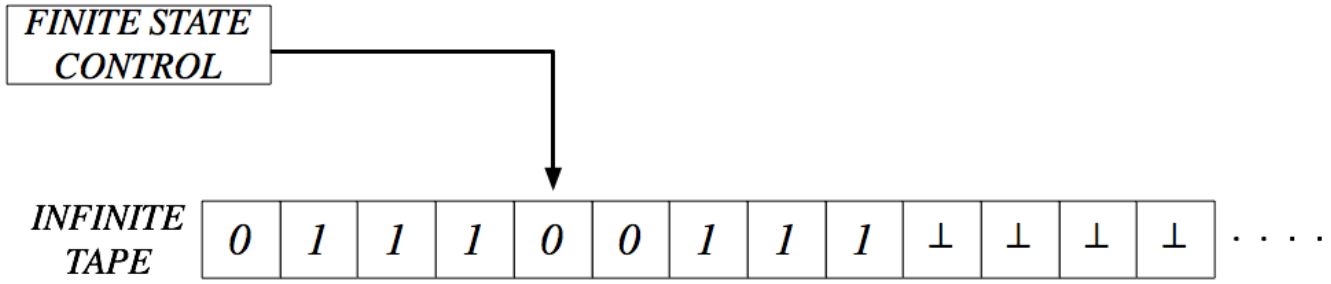


Turing Machines

- At each step the automaton:
  (1) is in some state
  (2) reads the symbol under the head
  (3) writes a symbol *(possibly the same one)* under the head
  (4) moves left or right
  (5) transitions to a new state *(possibly the same state)*

these depend only on (1,2)

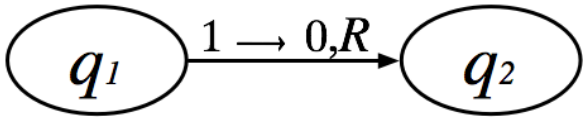
- Two special states: ***accept*** and ***reject***
  - When either of these are entered the machine halts





Turing Machines

- State diagram representation of Turing Machines:

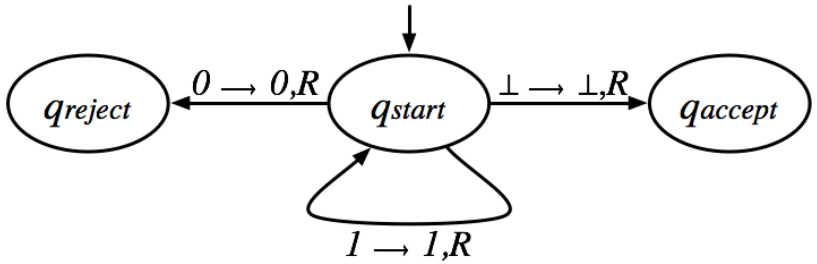


- Translation:
  - if you're in state q_1 and read a 1 under the head
  - write a 0, move to the right, and transition to state q_2



Turing Machines

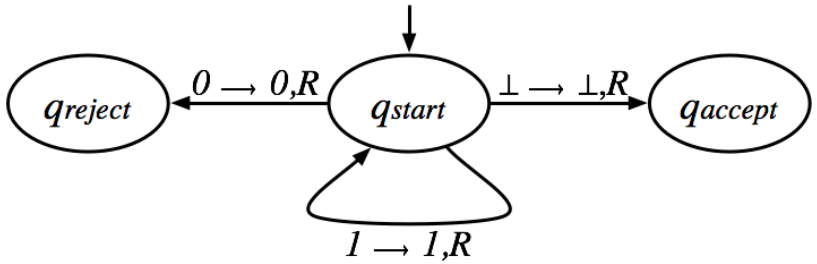
- What's the language of this TM?
- A TM recognizing L:

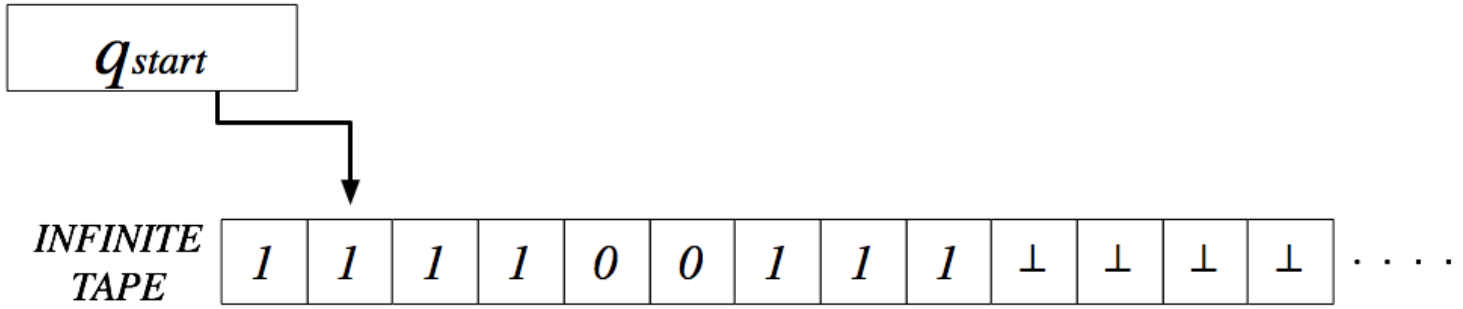




Turing Machines

- L = {w ∈{0,1}* | w contains no 0s}
- A TM recognizing L:

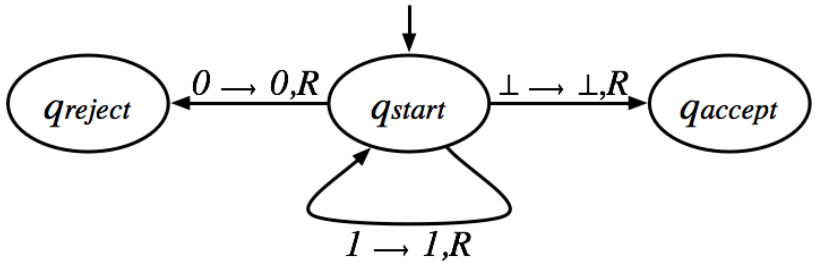


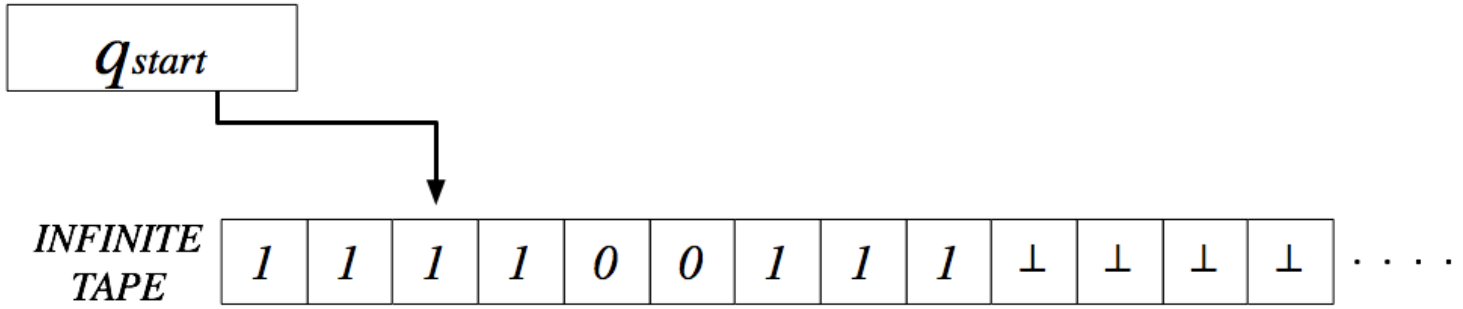




Turing Machines

- L = {w ∈{0,1}* | w contains no 0s}
- A TM recognizing L:

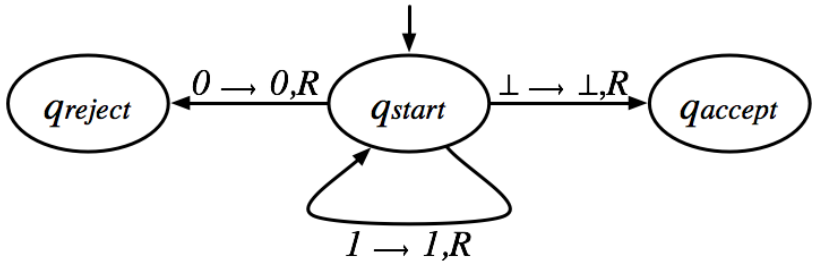


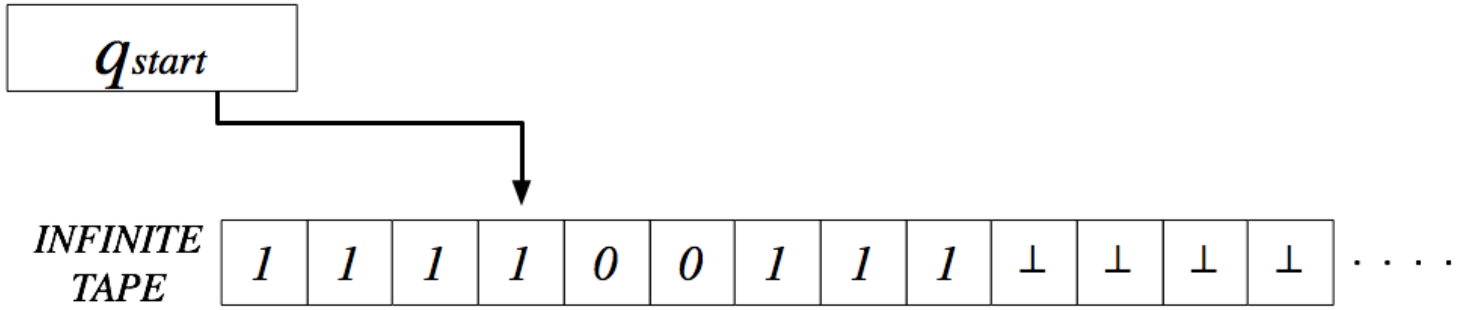




Turing Machines

- L = {w ∈{0,1}* | w contains no 0s}
- A TM recognizing L:

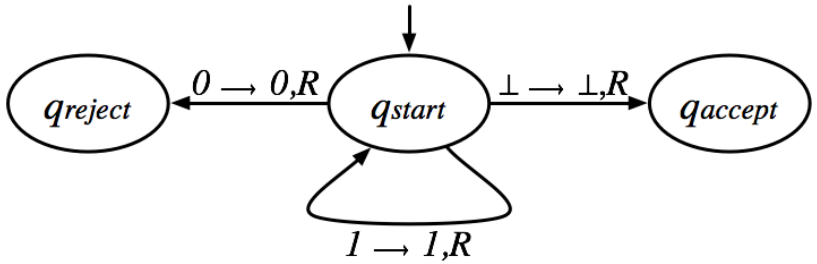


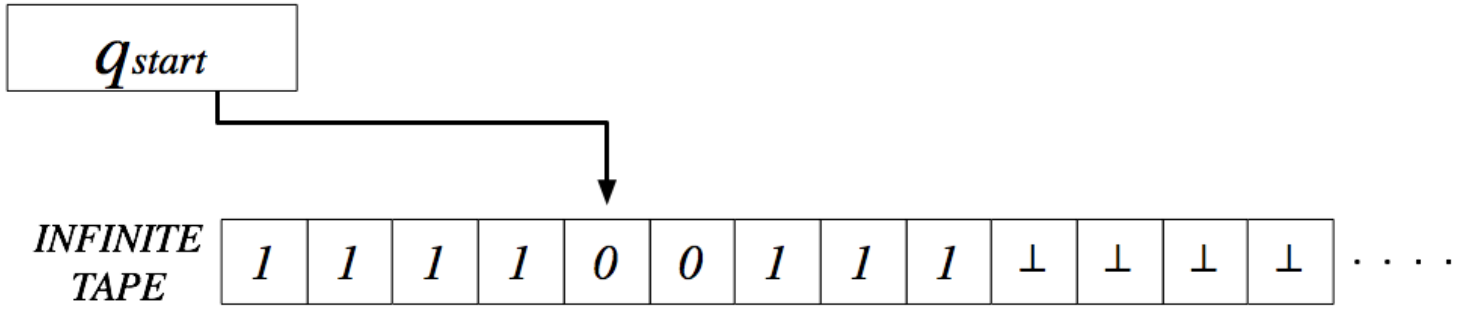




Turing Machines

- L = {w ∈{0,1}* | w contains no 0s}
- A TM recognizing L:

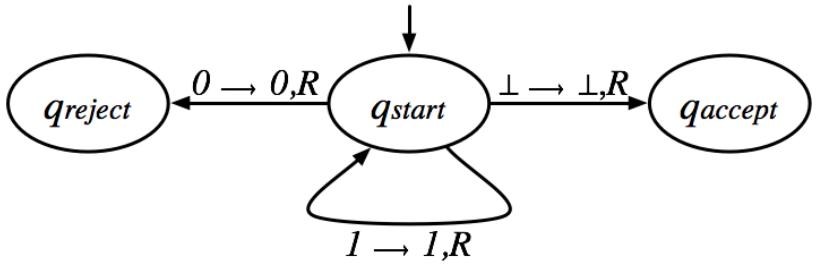


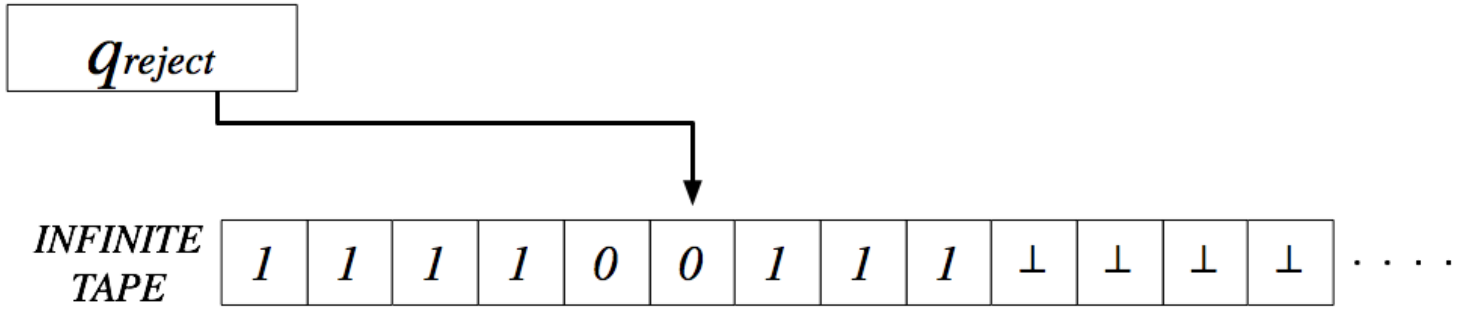




Turing Machines

- L = {w ∈{0,1}* | w contains no 0s}
- A TM recognizing L:







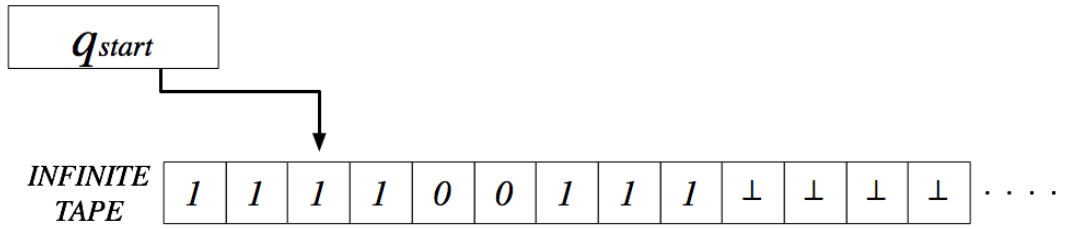
Formal Definition

- A Turing machine is a 7-tuple:
  - Q = set of ***states***
  - Σ = the ***input*** alphabet
  - Γ = the ***tape alphabet***, $\Sigma \cup \{\bot\} \subseteq \Gamma$
  - δ: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the ***transition function***
  - $q_{start} \in Q$
  - $q_{accept} \in Q$
  - $q_{reject} \in Q$, $q_{accept} \neq q_{reject}$



Semantics

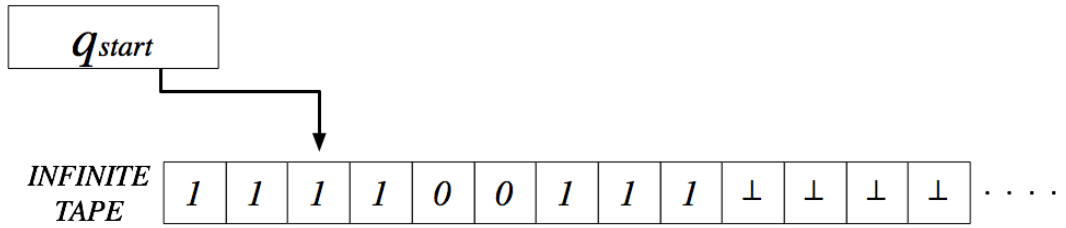
- A ***configuration*** is a list of the tape contents, head position, and state
  - e.g. configuration below is 11 q_{start} 1100111





Semantics

- A ***configuration*** is a list of the tape contents, head position, and state
  - e.g. configuration below is 11 $\mathbf{q_{start}}$ **1**1001111



- a,b∈**Γ**, **u**,**v**∈**Γ***
  - if **δ**(q_i,b) = (q_j,c,L), ***u*** a q_i b ***v*** yields ***u*** q_j a c ***v***
  - if **δ**(q_i,b) = (q_j,c,R), ***u*** a q_i b ***v*** yields ***u*** a c q_j ***v***
  - a ***halting configuration*** if $q_j \in \{q_{accept}, q_{reject}\}$



Languages of TMs

- For a T.M. M with input alphabet Σ:
- $L(M) = \{w \in \Sigma^* \mid$ *starting with w on the first |w| cells of the tape, M will eventually enter the accept state*$\}$

- If $w \notin L(M)$, what does M do when running on w?



Three outcomes

- ***NOTE:*** 3 different outcomes
  - The TM accepts (the input is in the TM's language)
  - The TM rejects
  - The TM runs forever

} input not in the language



Three outcomes

- ***NOTE:*** 3 different outcomes
  - The TM accepts (the input is in the TM's language)
  - The TM rejects
  - The TM runs forever

  } input not in the language

- A T.M. ***M*** recognizes a language ***A*** if
  - L(***M***) = ***A***
- A T.M. ***M*** decides a language ***A*** if
  - L(***M***) = ***A*** *and* ***M*** rejects all w ∉ ***A*** (i.e., it always halts)



Important

- Defn. M ***recognizes*** a language A if L(M) = A
- Defn. M ***decides*** A if L(M)=A and M *always* halts

- A language is ***recursively enumerable*** if some Turing machine recognizes it
- A language is ***recursive*** if some Turing machine decides it
- L is ***co-recognizable*** if $\bar{L}$ is recognizable



Important

- Defn. M ***recognizes*** a language A if L(M) = A
- Defn. M ***decides*** A if L(M)=A and M *always* halts.

- A language is ***recursively enumerable (or recognizable)*** if some Turing machine recognizes it
  - A Turing machine can enumerate all strings of the language
- A language is ***recursive (or decidable)*** if some Turing machine decides it
- L is ***co-recognizable*** if $\overline{L}$ is recognizable



Theorem

- A language is decidable iff it is recognizable and co-recognizable
  - If L is decidable, both L and $\bar{L}$ are recognizable
  - If L and $\bar{L}$ are recognizable by Turing machines M and M', respectively, then we can construct a new Turing machine which simulates M and M' in parallel

    If M accepts, *accept*

    If M' accepts, *reject*

In parallel means that we perform one step of M then one step of M'



Exercise

- Design a TM M that decides L(M) = { w#w | w ∈{0,1}*}



Exercise

- Design a TM M that decides L(M) = { w#w | w ∈{0,1}*}

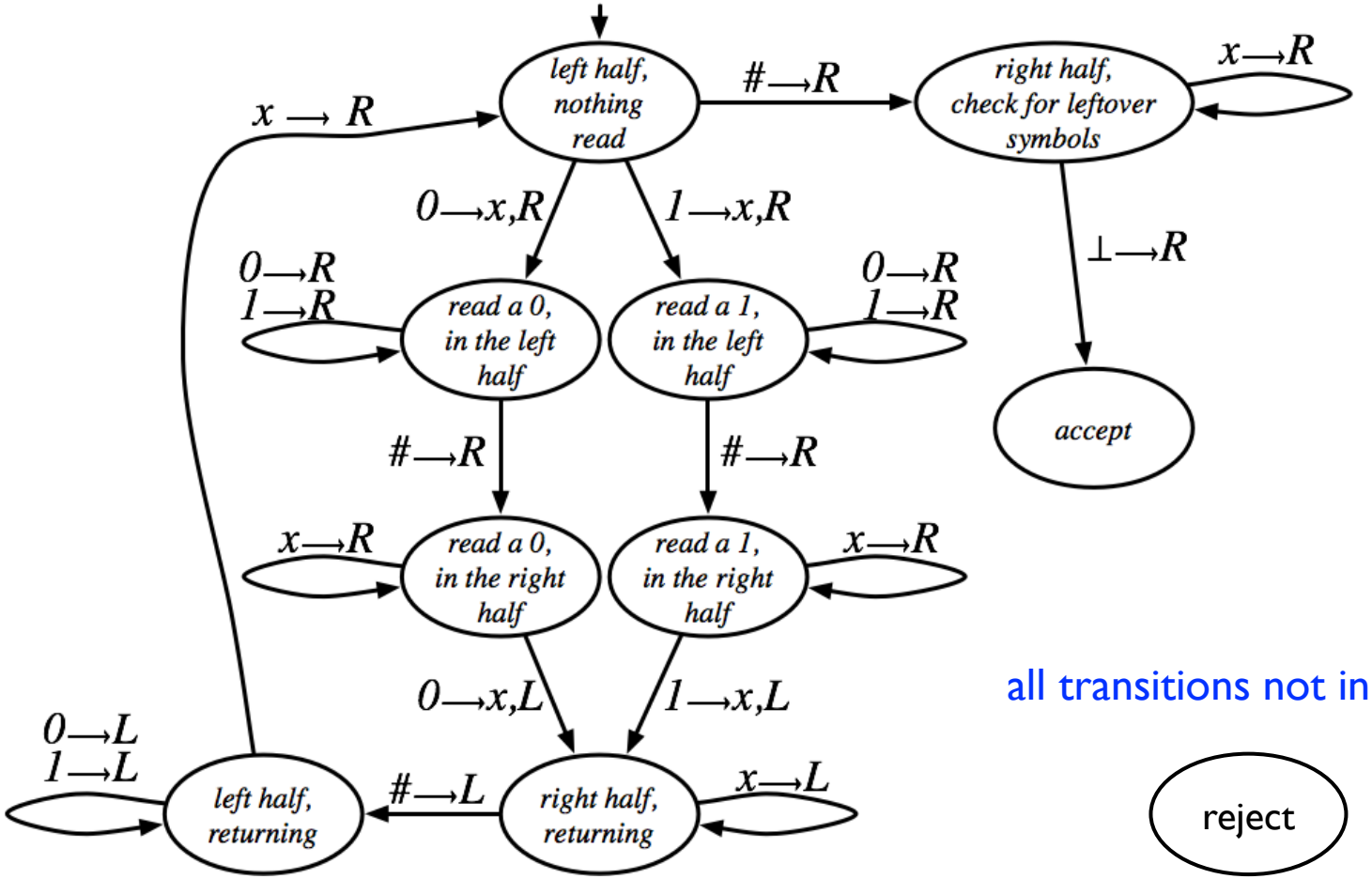
- Pseudocode:
  - Input = w#w'

    (Assume for simplicity that the input contains just one #)
  - For i = 1 .. |w|
    - Check if the ith bit of w = the ith bit of w'
    - If they're equal, mark them both "x"
  - If there are leftover bits (|w| < |w'|) **REJECT**
  - Otherwise **ACCEPT**





- A (possibly smallest) TM recognizing { w#w | w ∈ {0,1}*} :

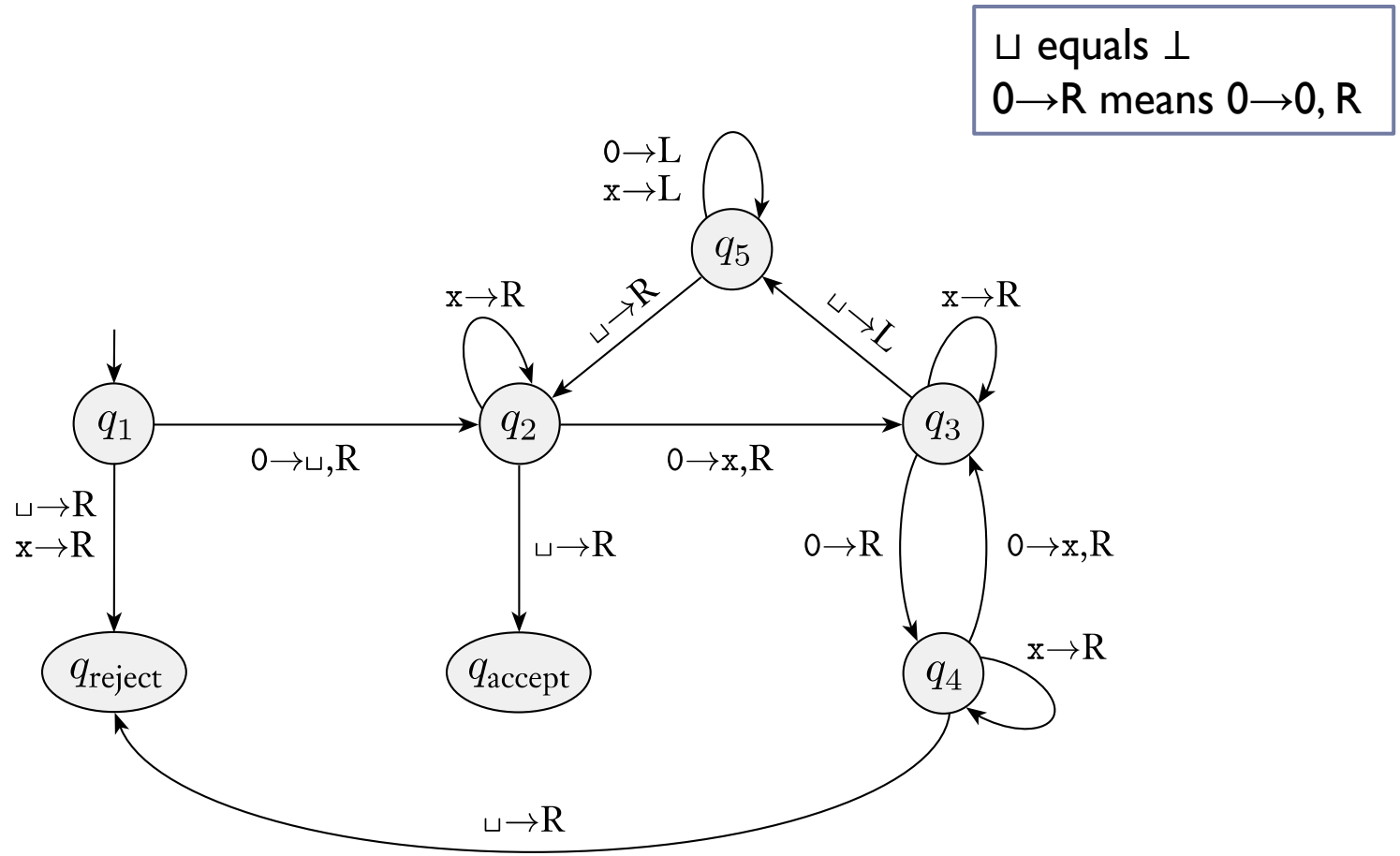


all transitions not in the diagram go to reject

reject

Example

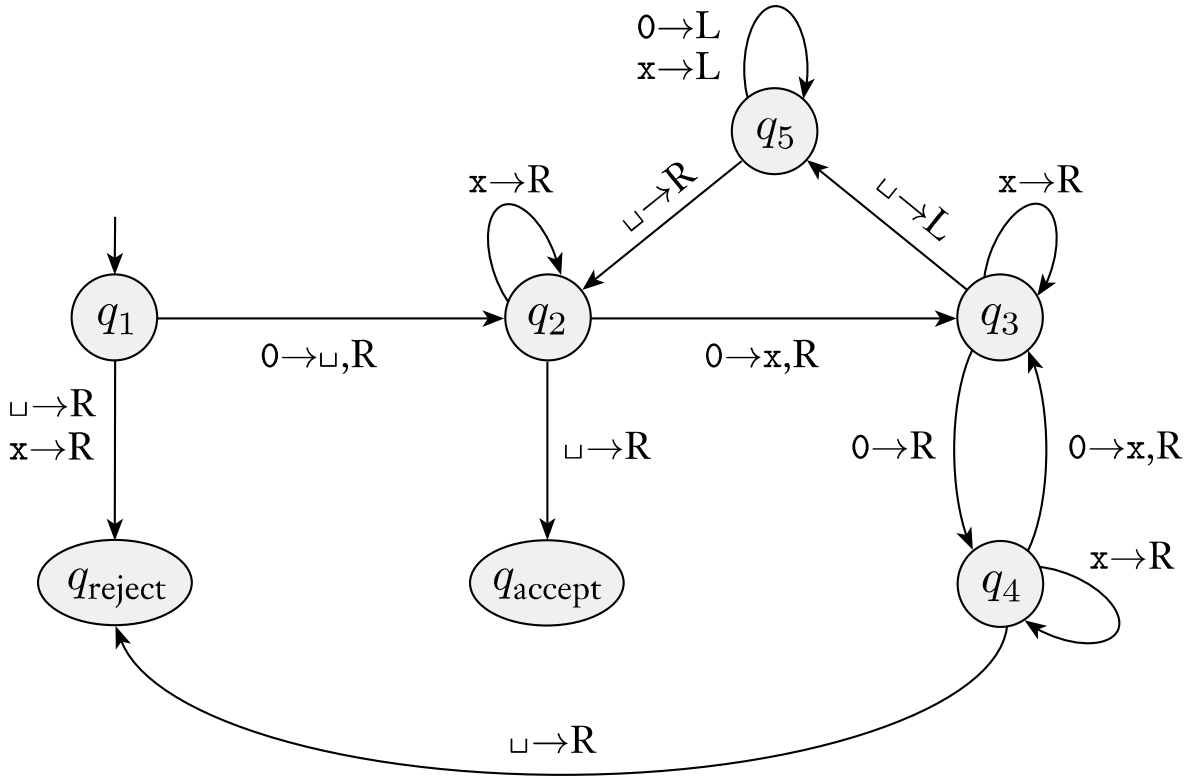
- What is the language of this TM?



Example

0	0	0	0	0	0	0	0	⊔
⊔	x	0	x	0	x	0	x	⊔
⊔	x	x	x	0	x	x	x	⊔
⊔	x	x	x	x	x	x	x	⊔

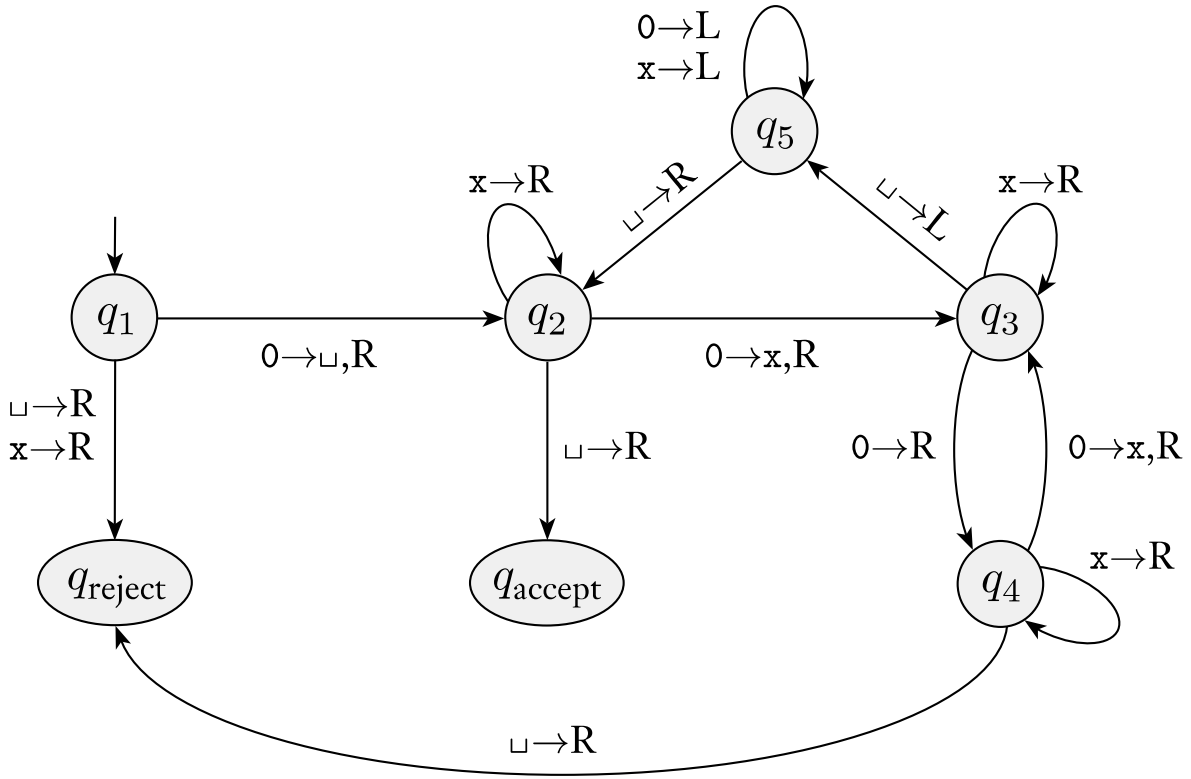
- What is the language of this TM?



Example

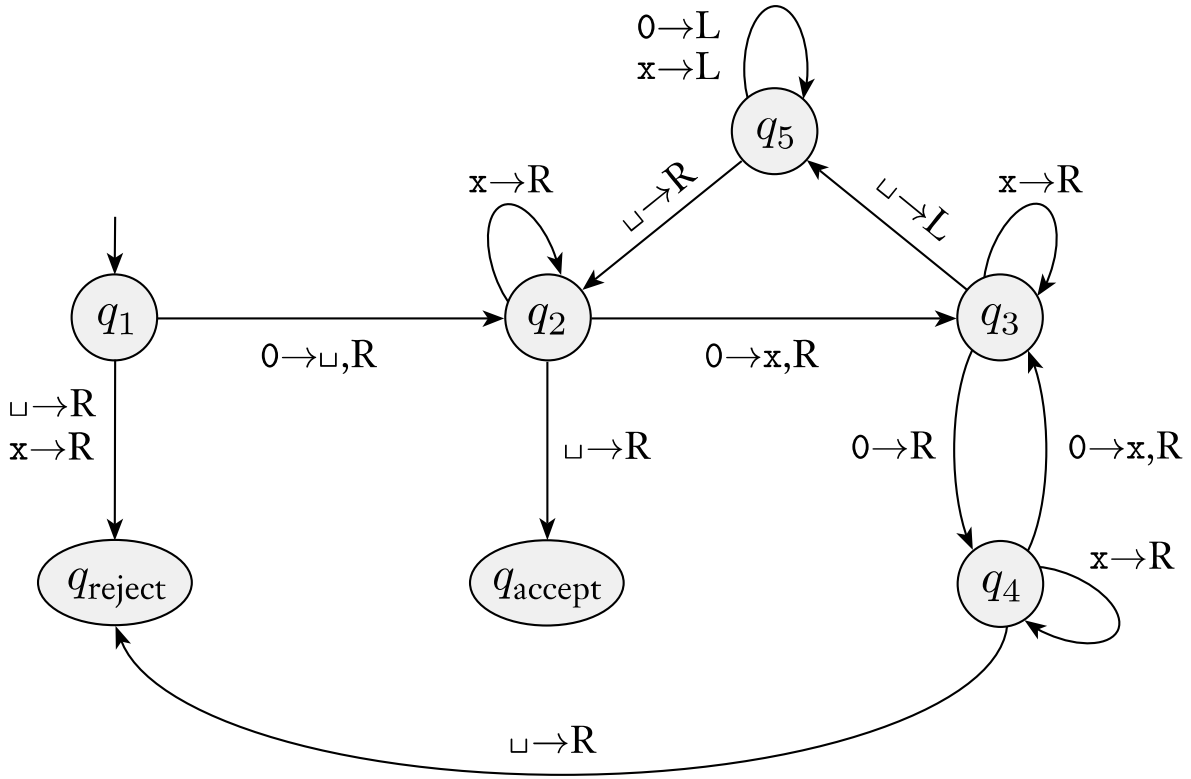
0	0	0	0	0	0	0	0	0	0	⊔
⊔	x	0	x	0	x	0	x	0	x	⊔
⊔	x	x	x	0	x	x	x	0	x	⊔

- What is the language of this TM?



Example

- What is the language of this TM?
  - $L = \{0^{2^n} \mid n \geq 0\}$



Why Turing Machines?

- A TM is not a good programming language!
  - Usually we don't need to write a real TM
  - Describing the algorithm is enough for describing a TM
  - TM is more convenient for theoretical proofs

- Turing tried to model the ***essence*** of computation (specifically: Turing himself working with a pencil and paper)



Why Turing Machines?

- A TM is not a good programming language!

  (But neither are many models of computation)

- Turing tried to model the ***essence*** of computation (specifically: Turing himself working with a pencil and paper)

  (1) Assume the paper is divided into discrete cells

  (2) Assume a finite number of symbols can be written in each cell

  (3) Assume the paper is infinite

  (4) Assume we can only look at one cell at a time

  (5) Assume we can only remember a fixed amount of information

  (6) Why 2 dimensional paper? A 1 dim. tape suffices



Church-Turing thesis

- A hypothesis about the nature of computable functions
- In simple terms, the Church–Turing thesis states that a function on the natural numbers is computable in an informal sense (i.e., computable by a human being using a pencil-and-paper method, ignoring resource limitations) if and only if it is computable by a Turing machine



Variants of Turing Machines

Formal Definition

- A Turing machine is a 7-tuple:
  - Q = set of ***states***
  - Σ = the ***input*** alphabet
  - Γ = the ***tape alphabet***, $\Sigma \cup \{\bot\} \subseteq \Gamma$
  - $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L,R\}$ is the ***transition function***
  - $q_{start} \in Q$
  - $q_{accept} \in Q$
  - $q_{reject} \in Q$, $q_{accept} \neq q_{reject}$



Variants of TMs

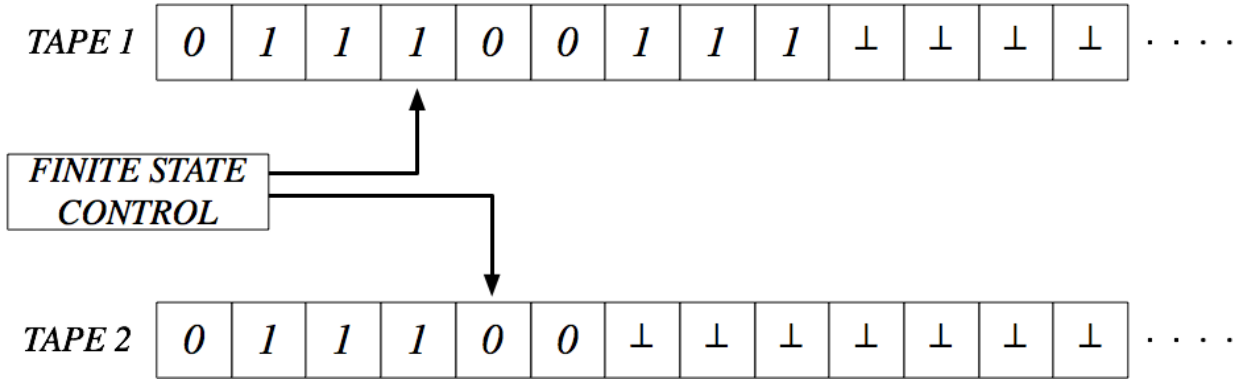
- Multiple tapes
- Nondeterministic choices (like DFAs vs. NFAs)
- TMs with output

- All with the same power!



2-Tape Turing Machine

- Same as a regular TM…
  - The input is written on Tape 1; Tape 2 initially blank
  - Reads a symbol off each tape, each head moves ***L***eft or ***R***ight or ***S***tays put



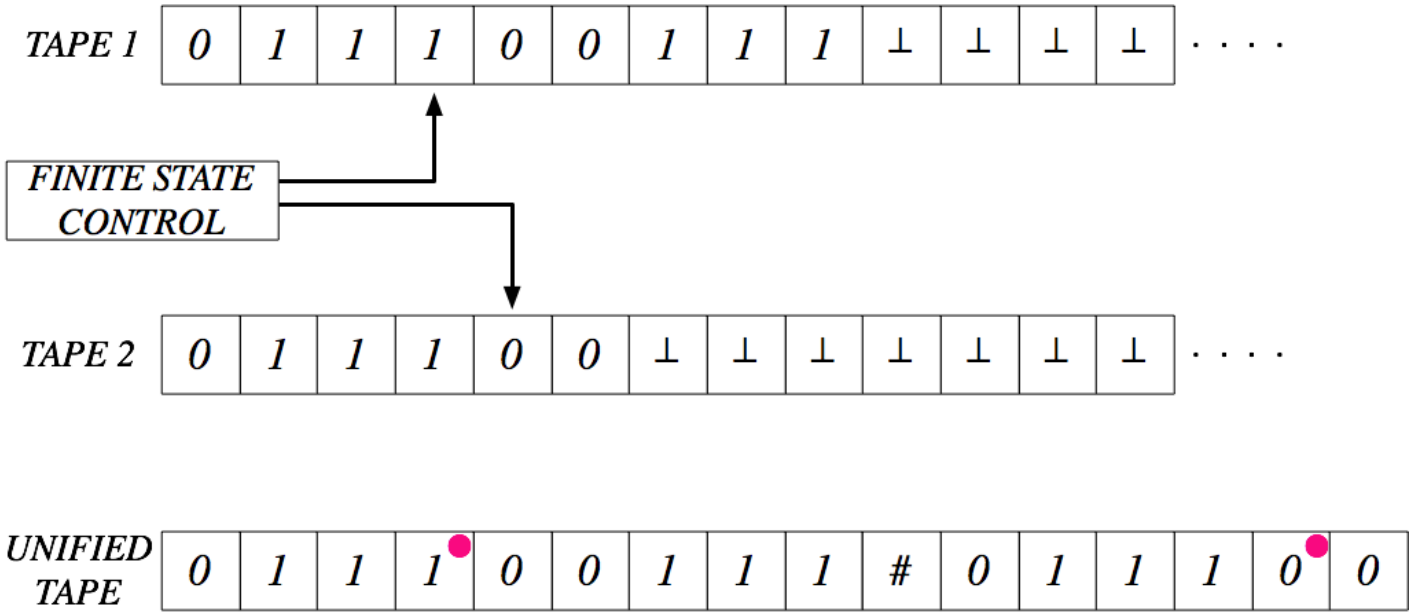


Multitape Turing Machines

- A k-tape Turing machine is a 7-tuple:
  - Q = set of ***states***
  - Σ = the ***input*** alphabet
  - Γ = the ***tape alphabet***, $\Sigma \cup \{\bot\} \subseteq \Gamma$
  - $\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L,R,S\}^k$ is the ***transition function***
  - $q_{start} \in Q$
  - $q_{accept} \in Q$
  - $q_{reject} \in Q$, $q_{accept} \neq q_{reject}$



- *Is it possible to simulate a 2-tape TM (or k-tape TM) on a standard TM?*



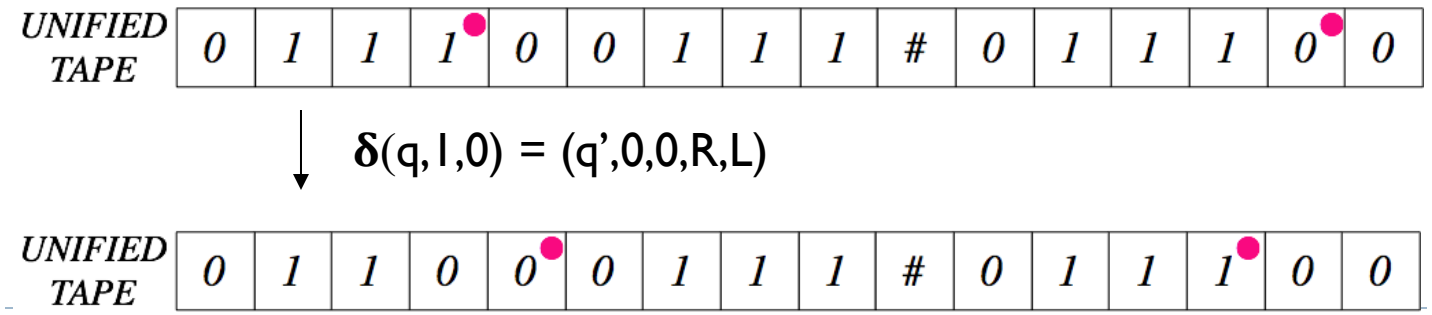
If the input alphabet is {0,1},
what's the tape alphabet?



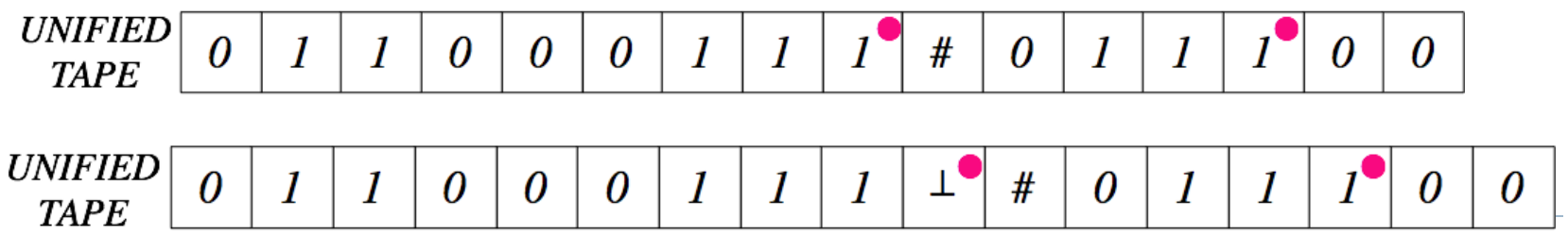
- Simulation Algorithm: first set everything up:
  - Replace the first symbol ***0/1*** with $\boldsymbol{0}^{\bullet}/\boldsymbol{1}^{\bullet}$
  - Write a '#' at the end of the input
  - Write a $\perp^{\bullet}$ after the #
  - Move the head back to the first cell



- Simulation algorithm:
  - Repeat: (the head is at the first cell marked •)
    - Note the symbol ***x*** • in the current cell
    - Move the head to the second •, note the symbol ***y*** • in that cell
    - Apply the transition $\delta(q,x,y) = (q',x',y',m,m')$ to the second • $m,m' \in \{L,R,S\}$
    - …then to the first
    - Stop if $q' = q_{accept}$ or q_{reject}



- One special case: if the left head tries to move to the #, shift # and all of tape 2 to the right



Theorem

- A language is Turing-recognizable if and only if some multitape Turing machine recognizes it
  - since this idea can be generalized to any number of tapes



Nondeterministic T.M.s

- Transition function gives set of possible next states, cell writes, L/R moves, etc
- i.e., it can "guess" the right thing to do

- The NTM accepts a string if there is ***SOME*** sequence of guesses that yields an accepting configuration



Nondeterministic Turing Machines

- An NTM is a 7-tuple:
  - Q = set of ***states***
  - Σ = the ***input*** alphabet
  - Γ = the ***tape alphabet***, $\Sigma \cup \{\bot\} \subseteq \Gamma$
  - $\delta: Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L,R\}}$ is the ***transition function***
  - $q_{start} \in Q$
  - $q_{accept} \in Q$
  - $q_{reject} \in Q$, $q_{accept} \neq q_{reject}$

How is this definition different from a standard TM?



Nondeterministic Turing Machines

- An NTM is a 7-tuple:
  - Q = set of ***states***
  - Σ = the ***input*** alphabet
  - Γ = the ***tape alphabet***, $\Sigma \cup \{\bot\} \subseteq \Gamma$
  - $\delta: Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L,R\}}$ is the ***transition function***
  - $q_{start} \in Q$
  - $q_{accept} \in Q$
  - $q_{reject} \in Q$, $q_{accept} \neq q_{reject}$

How is this definition different from a standard TM?

2^X = power set of X

Set of possible (next state, cell write, move) transitions



NTM Semantics – How are they different?

- $a,b \in \Gamma$, $\mathbf{u},\mathbf{v} \in \Gamma^*$
  - $\boldsymbol{u}\ a\ q_i\ b\ \boldsymbol{v}$ yields $\boldsymbol{u}\ q_j\ a\ c\ \boldsymbol{v}$ if $(q_j,c,L) \in \delta(q_i,b)$
  - $\boldsymbol{u}\ a\ q_i\ b\ \boldsymbol{v}$ yields $\boldsymbol{u}\ a\ c\ q_j\ \boldsymbol{v}$ if $(q_j,c,R) \in \delta(q_i,b)$
  - a ***halting configuration*** if $q_j \in \{q_{accept}, q_{reject}\}$
- w is accepted if there are configurations $C_1, C_2, \ldots, C_k$
  - $C_1 = q_{start}\ w$
  - C_i yields C_{i+1}, for $1 \le i < k$
  - C_k is a halting configuration in the accept state
- L(M) = the set of all strings accepted by the NTM M
- M ***decides*** L(M) if it ***always*** halts



NTM Semantics – How are they different?

(q_j,c,L) could be among many legal transitions

- $a,b \in \Gamma$, $u,v \in \Gamma^*$
  - $u\ a\ q_i\ b\ v$ yields $u\ q_j\ a\ c\ v$ if $(q_j,c,L) \in \delta(q_i,b)$
  - $u\ a\ q_i\ b\ v$ yields $u\ a\ c\ q_j\ v$ if $(q_j,c,R) \in \delta(q_i,b)$
  - a ***halting configuration*** if $q_j \in \{q_{accept}, q_{reject}\}$
- w is accepted if there are configurations $C_1, C_2, \ldots, C_k$
  - $C_1 = q_{start}\ w$
  - C_i yields C_{i+1}, for $1 \le i < k$
  - C_k is a halting configuration in the accept state
- L(M) = the set of all strings accepted by the NTM M
- M ***decides*** L(M) if it ***always*** halts



What “nondeterminism” really means

- “Your machine makes guesses, and it always guesses right”

- A non-deterministic algorithm is really a ***verification*** algorithm for strings in the language *(What is “guessing”?)*



“Programming” Nondeterminism

- L = {w ∈ {0,1}* | w contains a palindrome with length 3|w|/4}

- Sketch a simple nondeterministic TM recognizing L



“Programming” Nondeterminism

- L = {w ∈ {0,1}* | w contains a palindrome with length 3|w|/4}

- Sketch a simple nondeterministic TM recognizing L

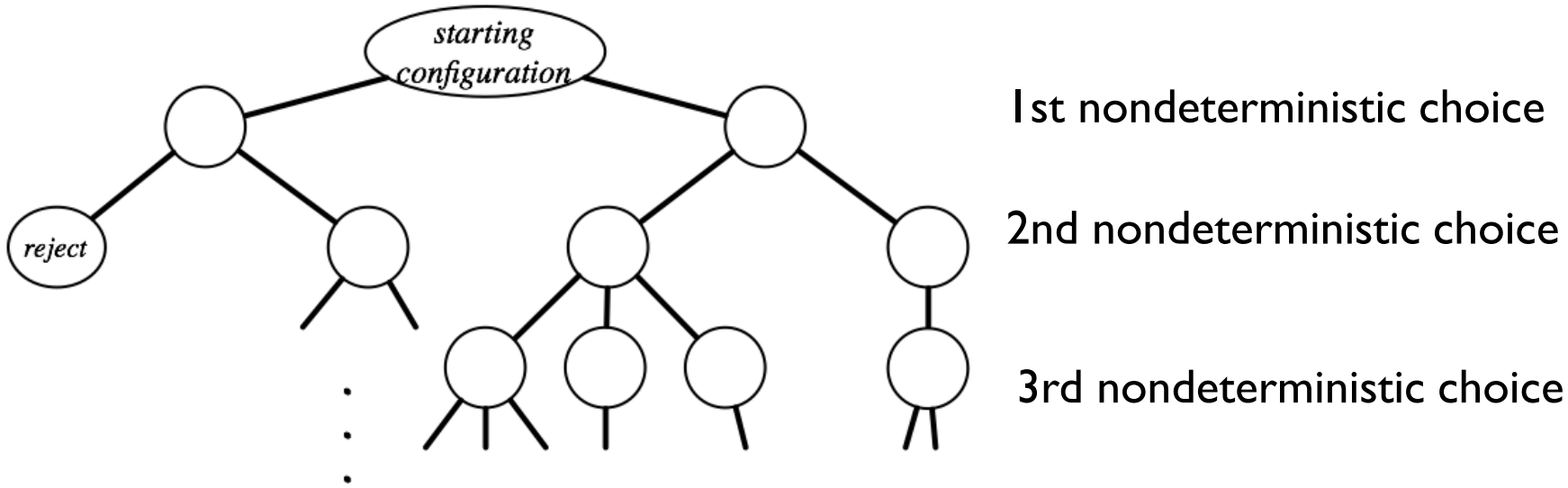
  (1) Pick |w|/4 cells and erase them

  (2) Check if the remaining cells form a palindrome



Simulating an NTM

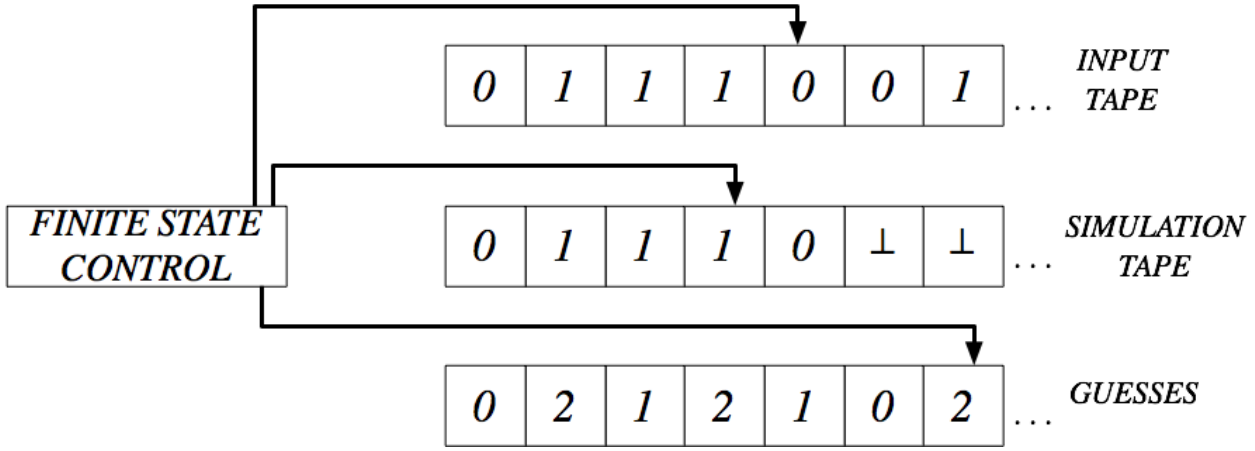
- Think of guesses forming a tree:





Simulate an NTM with a 3-tape TM

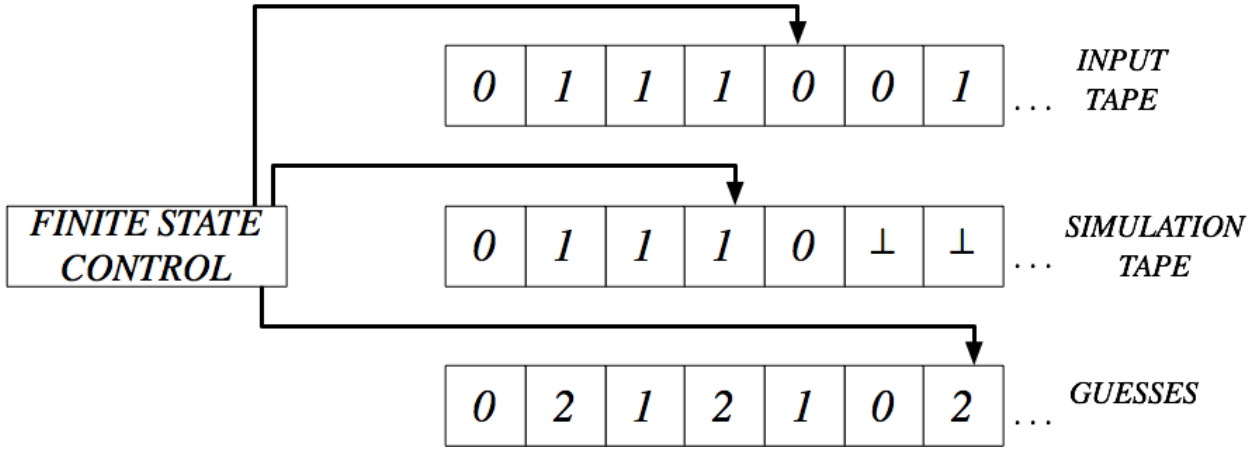
- 3-tape TM:
  - Tape 1 holds the input (don't write anything else here)
  - Tape 2 is a simulation tape
  - Tape 3 holds the nondeterministic guesses
- How should it work?





Simulate an NTM with a 3-tape TM

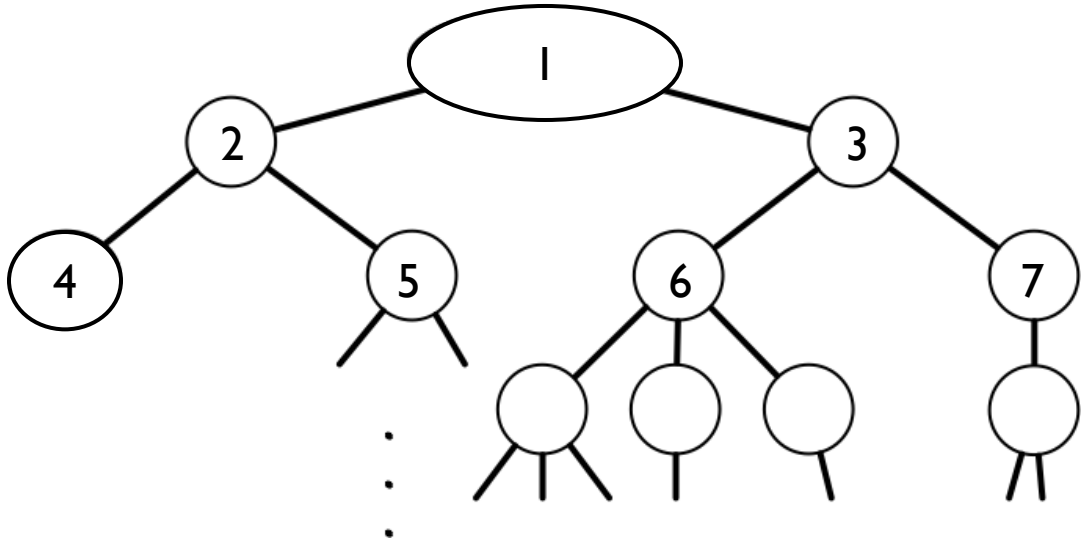
- 3-tape TM:
  - Tape 1 holds the input (don't write anything else here)
  - Tape 2 is a simulation tape
  - Tape 3 holds the nondeterministic guesses
- Depth-first search or Breadth-first search?



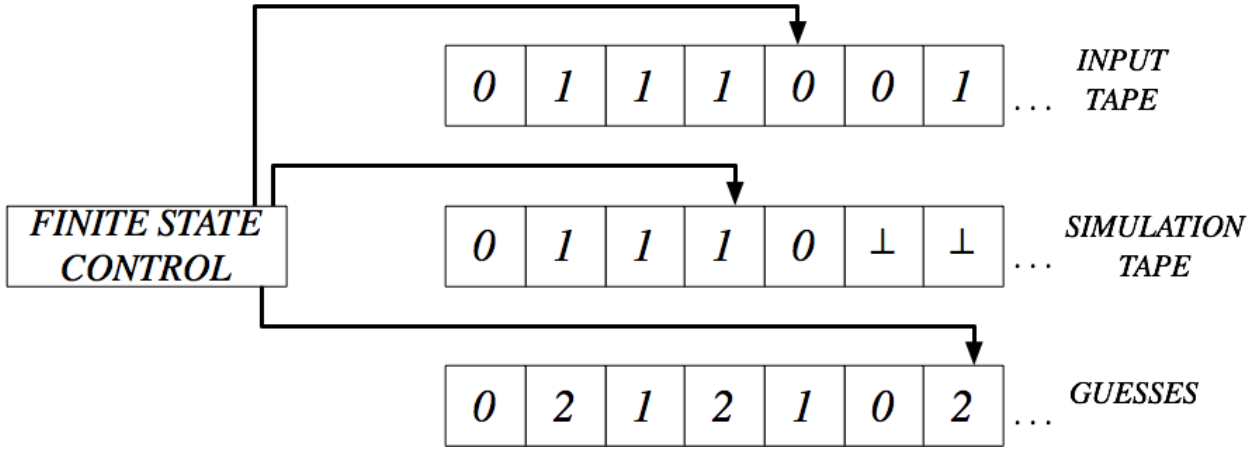


Simulate an NTM with a 3-tape TM

- 3-tape TM:
  - Tape 1 holds the input (don't write anything else here)
  - Tape 2 is a simulation tape
  - Tape 3 holds the nondeterministic guesses
- Depth-first search or Breadth-first search?



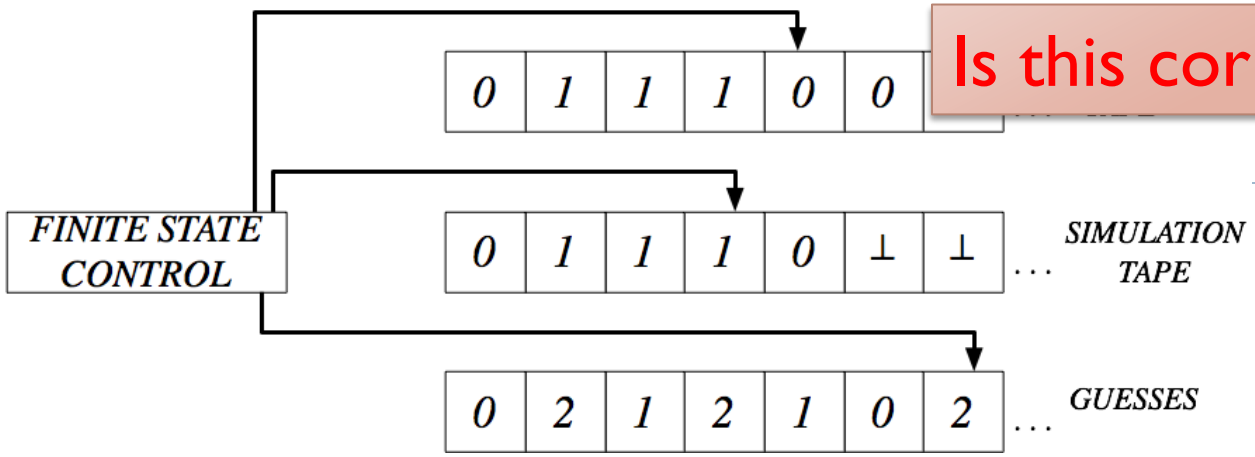




- Breadth-first search:
  1. Initially tape 1 contains the input, tapes 2 and 3 are empty
  2. copy tape 1 to tape 2
  3. Use tape 2 to simulate the NTM with the input. Use tape 3 as choices for all branches. *If no more symbols on tape 3, or if the nondeterministic choice is invalid, or the NTM reject, goes to step 4.* If it accepts, accept the input.
  4. Replace the string on tape 3 with the lexicographically next string. Go to step 2.

     (the string on tape 3 becomes longer and longer)





Is this correct?

- Breadth-first search:
  1. Initially tape 1 contains the input, tap
  2. copy tape 1 to tape 2
  3. Use tape 2 to simulate the NTM wit
     branches. *If no more symbols on tape 3* *or the NTM reject, goes to step 4.* If it accepts,
  4. Replace the string on tape 3 with the lexicographically next string. Go to step 2.

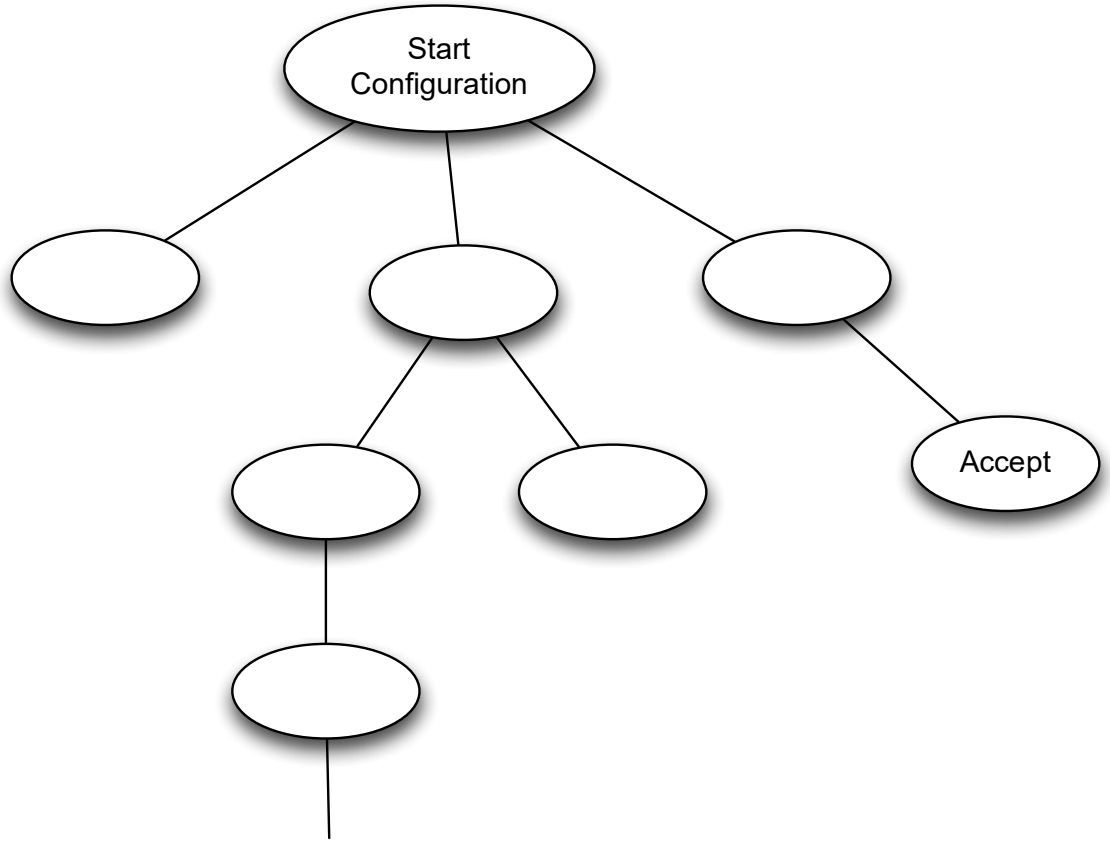
     (the string on tape 3 becomes longer and longer)

Lexicographic ordering is the dictionary ordering except the shorter string precedes longer string:

e.g. for $\Sigma=\{0,1\}$
$\{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, \ldots\}$

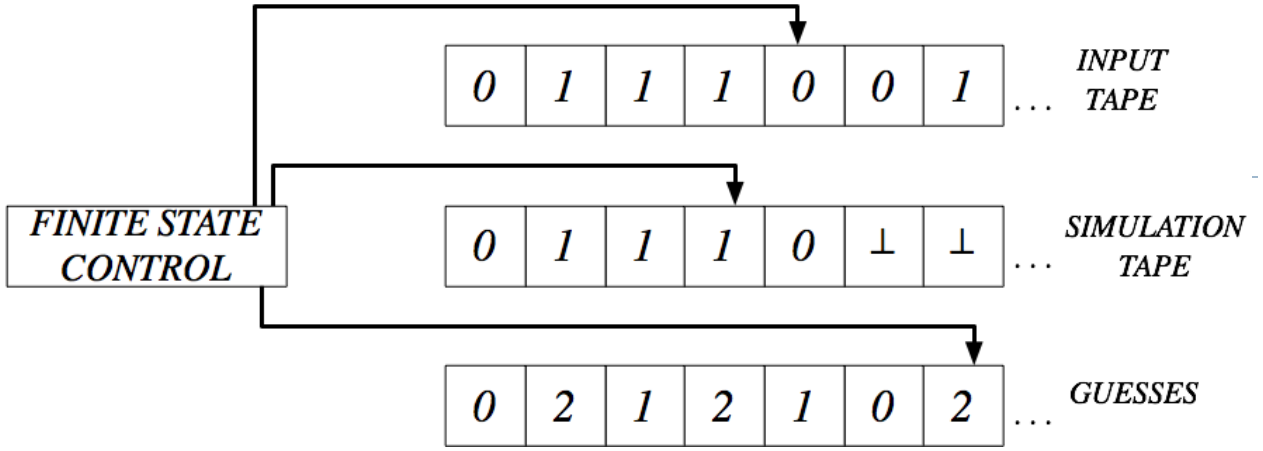
How to avoid…?

- The search goes into an infinite loop although the original NTM accepts



. infinite loop without nondeterministic guesses
.
.





This guess is for every step!! Including those "deterministic steps" (only one guess can be correct)

Breadth-first search:

1. Initially tape 1 contains the input, tapes 2 and 3 are empty
2. copy tape 1 to tape 2
3. Use tape 2 to simulate the NTM with the input. Use tape 3 as choices for all branches. *If no more symbols on tape 3, or if the nondeterministic choice is invalid, or the NTM reject, goes to step 4.* If it accepts, accept the input.
4. Replace the string on tape 3 with the lexicographically next string. Go to step 2.

   (the string on tape 3 becomes longer and longer)



- A language is Turing-recognizable if and only if some nondeterministic Turing machine recognizes it

- Is the exponential blowup of time necessary?
  - We have seen that NFA to DFA of exponential size
  - but we don't know whether NP ?= P



- A language is Turing-recognizable if and only if some nondeterministic Turing machine recognizes it

- We call a nondeterministic Turing machine a decider if all branches halt on all inputs



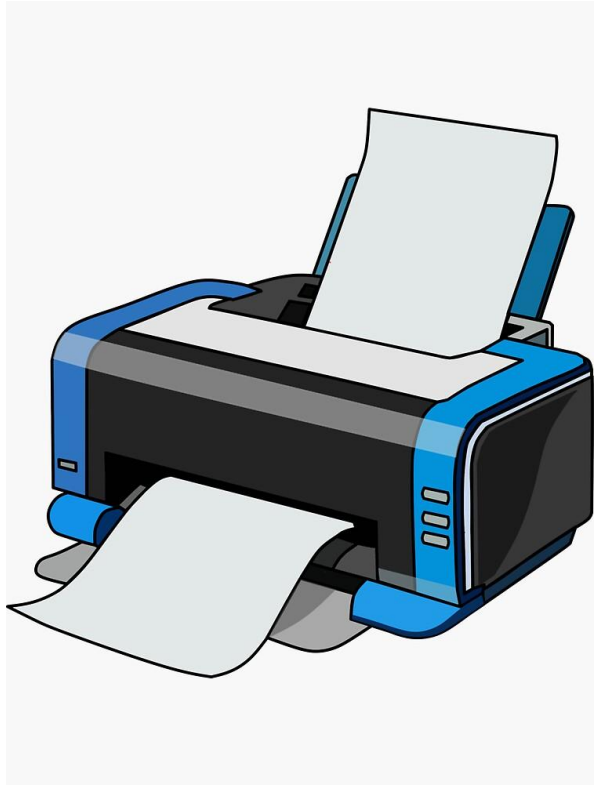
- A language is decidable if and only if some nondeterministic Turing machine decides it.
- If the NTM accepts w, then the simulator accepts
- If the NTM rejects w, how to modify the simulator so that it rejects w?
  - On the "branch tape", we won't try strings whose prefix yields accept or reject, so only finite number of strings is enough
  - When for some length L, all guesses yield accept or reject, halt



TM with outputs

- can be seen as multiple TMs

- TM with an extra "write-only tape"
  - Head can only move to the right
  - or a "printer": whenever print, cannot erase





Enumerator

- A Turing machine with a printer
  - It can enumerate strings one by one, maybe with repetitions
  - doesn't halt

- Why Turing-recognizable language is called recursively enumerable?



- A language is Turing-recognizable if and only if some enumerator generates it



- A language is Turing-recognizable if and only if some enumerator generates it

- If we have an enumerator E generating L, make TM M with input w:
  - Run E, every time that E outputs a string, compare it with w
  - if w appears in the outputs of E, accept



- A language is Turing-recognizable if and only if some enumerator generates it

- If a TM M recognizes a language L, make the enumerator E:

```
Ignore inputs of E
For all strings s
    simulate M on input s
    if accept, output s
```

What's wrong?



- A language is Turing-recognizable if and only if some enumerator generates it

- If a TM M recognizes a language L, make the enumerator E:

```
Ignore inputs of E
represents the strings by natural numbers
For i=1,2,3,... ,
        For j=0,1,...,i,
                Run the first i steps of M(j)
                If accept, output the string represented by j
```

  - eventually we can reach any M(j) which takes i (i>>j) steps to accept j



Two great ideas from Turing:

- We can encode Turing machines to natural numbers
  - So the set of Turing machines is countable

- Universal Turing machine:
  - a Turing machine that can simulate an arbitrary Turing machine on arbitrary input
  - inputs of U: coding of a TM <M> and its input w



Two great ideas from Turing:

- We can encode Turing machines to natural numbers
  - So the set of Turing machines is countable
  - Nowadays, everything is numbers (digital): text, audios, videos, programs, …

- Universal Turing machine:
  - a Turing machine that can simulate an arbitrary Turing machine on arbitrary input
  - Every modern computer is universal: what it does depends on the software, not hardware

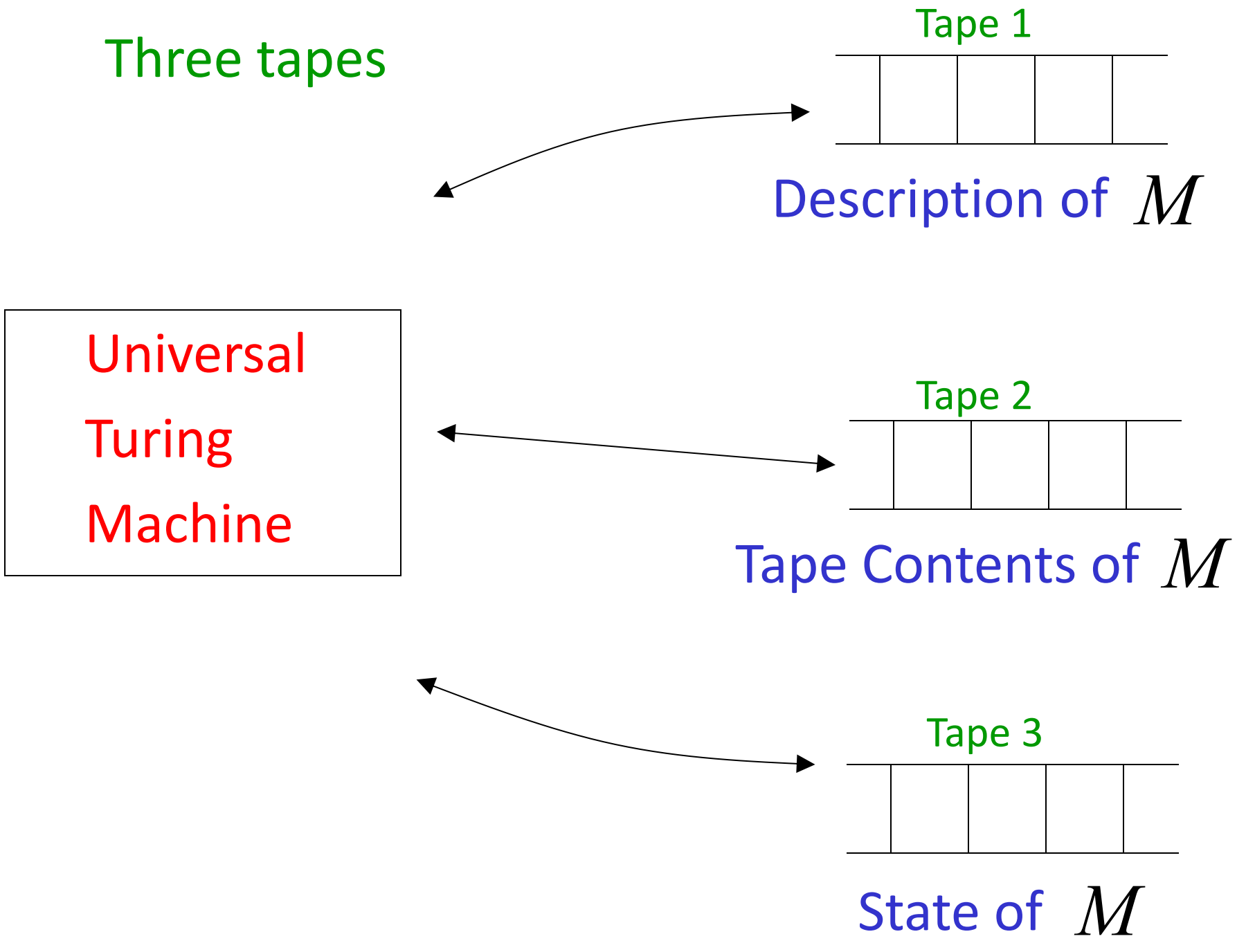


Universal Turing Machines

- Find a ***single*** TM M_{univ} that can simulate ***any*** TM M
- The input to M_{univ} is <M,w>
  - M_{univ} **accepts** <M,w> $\Leftrightarrow$ M **accepts** w
  - M_{univ} **rejects** <M,w> $\Leftrightarrow$ M **rejects** w
  - M_{univ} **loops** on <M,w> $\Leftrightarrow$ M **loops** on w

- How does M_{univ} work? (Give yourself a few tapes)




Three tapes
Tape 1
Description of M
Universal
Turing
Machine
Tape 2
Tape Contents of M
Tape 3
State of M

Fine, but is there any ***money*** in it?

- ***Yes actually!*** Stephen Wolfram $25,000



- What is the ***smallest*** universal TM? (Minsky, 1962)

- Answered by this guy Alex Smith

  There is a universal TM with 2 states and a 3-symbol alphabet



Halting Problem

The Language A_{TM}

- A_{TM} = {<M,w> | M is a TM and M accepts w}

- A_{TM} is a ***recognizable*** language *(Why?)*



The Language A_{TM}

- A_{TM} = {<M,w> | M is a TM and M accepts w}

- A_{TM} is a ***recognizable*** language
  - Simulate M on input w
  - If it accepts, accept
  - If it rejects, reject
  - (If M loops, the simulation will also loop)

Can we distinguish between these two scenarios?

- A_{TM} is ***not*** a Turing-***decidable*** language
  - we need to prove this!

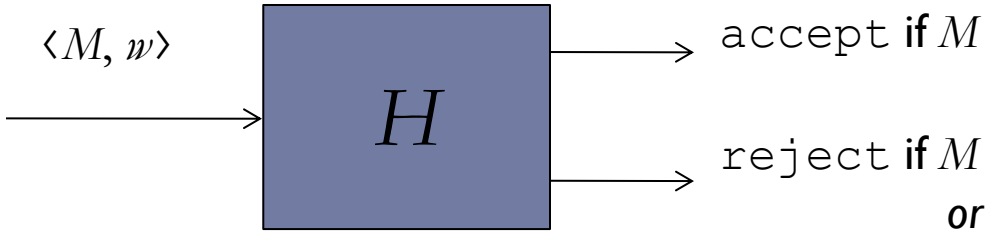


Proof of undecidability

- Proof by contradiction:

Suppose A_{TM} is decidable

Then there is a TM H that decides A_{TM}:



What happens when $w = \langle M \rangle$?

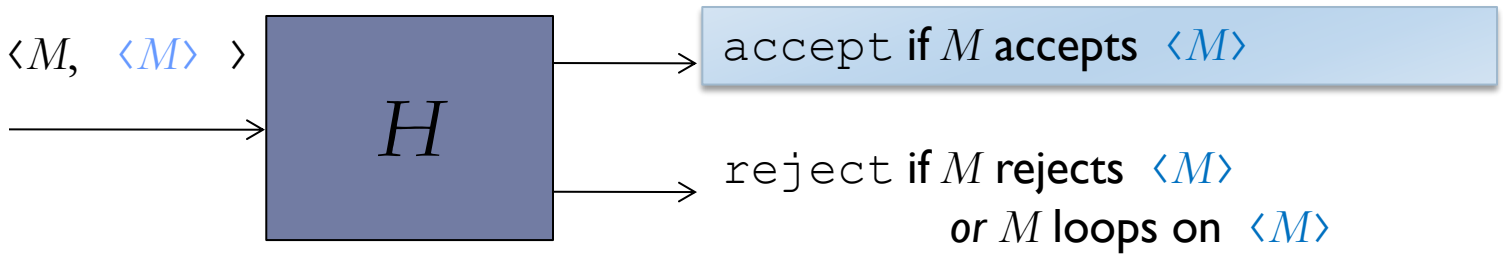


Proof of undecidability

- Proof by contradiction:

Suppose A_{TM} is decidable

Then there is a TM H that decides A_{TM}:



What happens when $w = \langle M \rangle$?



- We write a procedure D with input <M>:
  - D always halts, since H always halts

```
D(<M>):
        (<M> is the description of a TM)
        If H(<M,<M>>) accepts
                then reject;
                else accept;
```



- We write a procedure D with input <M>:
  - D always halts, since H always halts

```
D(<M>):
        (<M> is the description of a TM)
        If H(<M,<M>>) accepts
            then reject;
            else accept;
```

- Consider D(<D>),
  - if D(<D>) accepts, H(<D,<D>>) will accept, then contradicts



- We write a procedure D with input <M>:
  - D always halts, since H always halts

```
D(<M>):
        (<M> is the description of a TM)
        If H(<M,<M>>) accepts
                then reject;
                else accept;
```

- Consider D(<D>),
  - if D(<D>) rejects, H(<D,<D>>) will reject, then contradicts

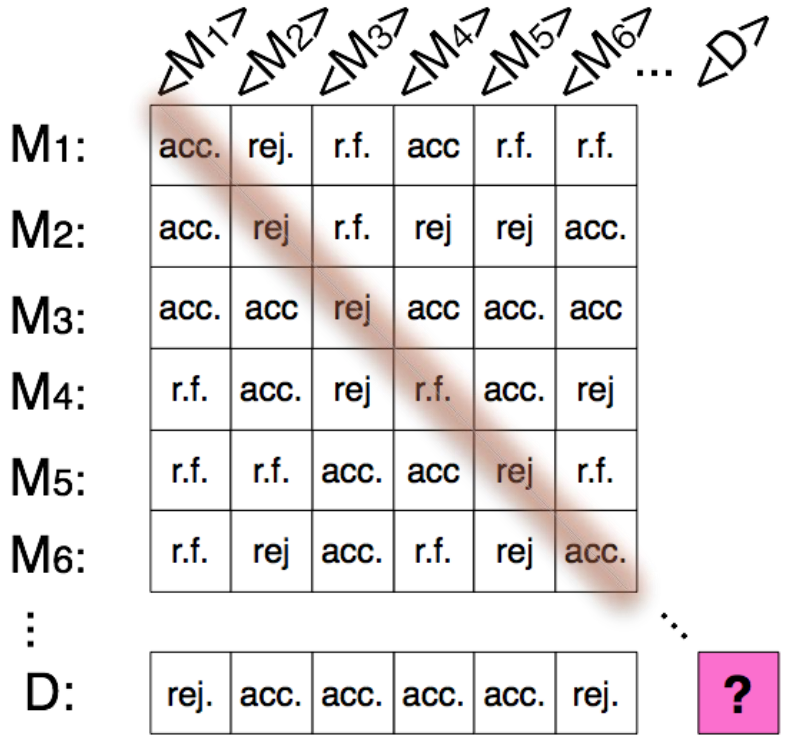


A_{TM} is Undecidable

- D(<M_i>) will be different from M_i(<M_i>)
- What can D do on input <D>?
  - *It can't run forever*

    H always halts so D halts too

    (they're deciders)
  - *Can it accept or reject?*



Unrecognizable languages

The language A_{TM} is recognizable but not decidable



Unrecognizable languages

The language A_{TM} is recognizable but not decidable

- Reminder: A language is decidable iff it is recognizable and co-recognizable
- Thus, A_{TM} is not co-recognizable



Unrecognizable languages

The language A_{TM} is recognizable but not decidable

- How about languages that are not recognizable?

$$\overline{A_{\text{TM}}} = \{ \langle M, w \rangle : M \text{ is a TM that does not accept } w\}$$

$$= \{ \langle M, w \rangle : M \text{ rejects or loops on input } w\}$$

The language $\overline{A_{\text{TM}}}$ is not recognizable



The Halting Problem

- $HALT_{TM}$ = {<M,w> | TM M halts on input w>

- If you can *decide* $HALT_{TM}$ then you can *decide* A_{TM} *(Why?)*



The Halting Problem

- $HALT_{TM}$ = {<M,w> | TM M halts on input w>

- If you can *decide* $HALT_{TM}$ then you can *decide* A_{TM}
  - Suppose there's a machine H that decides $HALT_{TM}$
  - Then G will decide A_{TM}

```
G(<M,w>)
    if H(<M,w>) rejects
            then reject;
            else return M(<w>);
```



The Halting Problem

- $HALT_{TM}$ = {<M,w> | TM M halts on input w>

- If you can *decide* $HALT_{TM}$ then you can *decide* A_{TM}
  - Suppose there's a machine H that decides $HALT_{TM}$
  - Then G will decide A_{TM}

```
G(<M,w>)
    if H(<M,w>) rejects
            then reject;
            else return M(<w>);
```

  - But, we have known A_{TM} is not decidable
  - Thus, $HALT_{TM}$ is not decidable



Oracle Turing Machines

- Oracle Turing Machine (OTM):
  - multitape TM M with special "query" tape
  - and special states q_{query}, q_{yes}, q_{no}
  - on input x, with oracle language A
  - M^A runs as usual, except…
  - when M^A enters state q_{query}:
    - y = contents of query tape
    - $y \in A \Rightarrow$ transition to q_{yes}
    - $y \notin A \Rightarrow$ transition to q_{no}



Turing Reducibility

- Suppose you have an "oracle" which decides a language B
- Oracle Turing machine: a TM capable of querying an oracle
- M^B: the oracle TM that has an oracle for language B

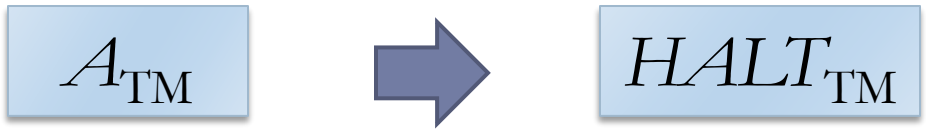


Turing Reducibility

- Suppose you have an "oracle" which decides a language B
- Oracle Turing machine: a TM capable of querying an oracle
- M^B: the oracle TM that has an oracle for language B
- Language A is Turing reducible to B, if an oracle TM M^B decides A, write $A \leq_T B$



- We just assume there is an oracle for HALT_{TM}
- Then we can solve A_{TM}, so A_{TM} is "no harder" than HALT_{TM}



- From A_{TM} is undecidable, HALT_{TM} is also undecidable



Collatz conjecture

- This program always halts:

```
int collatz(int n) {
        while(n>1) n = (n%2) ?  3n+1 : n/2;
}
```

```
n= 27
27 82 41 124 62 31 94 47 142 71 214 107 322 161 484 242 121 364 182 91 274 137 412 206 103 310 155
466 233 700 350 175 526 263 790 395 1186 593 1780 890 445 1336 668 334 167 502 251 754 377 1132 566
 283 850 425 1276 638 319 958 479 1438 719 2158 1079 3238 1619 4858 2429 7288 3644 1822 911 2734 13
67 4102 2051 6154 3077 9232 4616 2308 1154 577 1732 866 433 1300 650 325 976 488 244 122 61 184 92
46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2
```

- The conjecture has been shown to hold for all positive integers that have been tried, up to a very large number: 2.95×10^{20}, but no general proof has been found
- Paul Erdős said about the Collatz conjecture: "Mathematics may not be ready for such problems"



Proof of undecidability

- Prove B is undecidable: reduce an undecidable problem A to B

- We have known
  - A_{TM} = {<M,w> | M is a TM and M accepts w}
  - $HALT_{TM}$ = {<M,w> | TM M halts on input w}
- are undecidable



Example 1

$$SOME_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts some input} \}$$

decidable

undecidable

- Step 1: You gotta believe it
  - To know if M is in $SOME_{\text{TM}}$, looks like we have to simulate
  - But then we might end up in a loop
- Step 2: Prove it!



Example 1: Implementing the reduction

Task Given $\langle M, w \rangle$, construct M' so that:

If M accepts w, then M' accepts some input

If M does not accept w, then M' accepts no inputs

M' := On input z,

Simulate M on input w
If M accepts, *accept*
Otherwise, *reject*



Kleene's *s-m-n* Theorem

- Suppose we have a TM $M(x_1,\ldots,x_m, y_1,\ldots,y_n)$ whose input is composed of m+n variables

- There is a TM $T(\langle M\rangle, x_1,\ldots,x_m)$ when given $M, x_1,\ldots,x_m$, outputs a TM M' such that:
  - $M'(y_1,\ldots,y_n)= M(x_1,\ldots,x_m, y_1,\ldots,y_n)$
  - (Simply put $x_1,\ldots,x_m$ into the machine M)

Note that the output of T is a Turing machine



Example 1: Writing it up

undecidable

$SOME_{TM} = \{ \langle M \rangle : M$ is a TM that accepts some input$\}$

$A_{TM} = \{ \langle M, w \rangle : M$ is a TM that accepts input $w\}$

Suppose $SOME_{TM}$ is decidable and R decides it. Let

$S :=$ On input $\langle M, w \rangle$ where M is a TM and w is a string

Construct the following TM M':

$M' :=$ On input x,
Simulate M on input w and output answer

Run R on input $\langle M' \rangle$ and output its answer

Then S accepts $\langle M, w \rangle$ if and only if M accepts w

So S decides A_{TM}, which is impossible



Example 2

$$E_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts no input} \}$$

decidable



Show: If E_{TM} can be decided by some TM R,

... then $SOME_{\text{TM}}$ can be decided by another TM S

$$SOME_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts some input} \}$$



Example 2

undecidable

$$E_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts no input} \}$$

$$SOME_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts some input} \}$$

Suppose E_{TM} can be decided by some TM R

Consider the following TM S:

> S := On input $\langle M \rangle$ where M is a TM
>
> Run R on input $\langle M \rangle$
>
> If R accepts, *reject*. If R rejects, *accept*.



Example 3

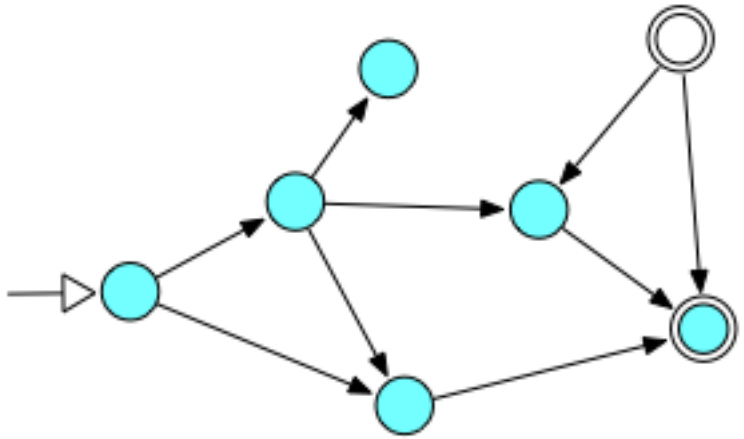
$$E_{\text{DFA}} = \{\ \langle D \rangle\ : D \text{ is a DFA and } L(D) \text{ is empty}\}$$



Example 3

$$E_{\text{DFA}} = \{\ \langle D \rangle\ : D \text{ is a DFA and } L(D) \text{ is empty}\}$$

- Just consider the reachability from start state to accept states:





Example 3

$$E_{\text{DFA}} = \{ \langle D \rangle : D \text{ is a DFA and } L(D) \text{ is empty} \}$$

decidable

- The following TM decides E_{DFA}:

$T :=$ On input $\langle D \rangle$, where D is a DFA
Mark the start state of D
Repeat
If $\delta(q,a)=q'$ for some a where q is marked
then mark q'
Until no new state can be marked
If no accept state is marked, accept; otherwise, reject



Example 4

$$E_{\text{CFG}} = \{ \langle G \rangle : G \text{ is a CFG and } L(G) \text{ is empty} \}$$



Example 4

$$E_{\text{CFG}} = \{ \langle G \rangle : G \text{ is a CFG and } L(G) \text{ is empty} \}$$

decidable

- The following TM decides E_{CFG}:

On input $\langle G \rangle$, where G is a CFG

Mark all terminal symbols on G

Repeat

For a rule $A \rightarrow U_1U_2 \ldots U_k$, if $U_1U_2 \ldots U_k$ has been marked, mark A

Until no new variables get marked

If the start variable is not marked, accept; otherwise reject



Example 5

$$L = \{ \langle \mathrm{M} \rangle : L(M) \text{ is regular} \}$$



Example 5

$$L = \{ \langle M \rangle : L(M) \text{ is regular} \}$$

- we have known that even

$$E_{TM} = \{ \langle M \rangle : L(M) = \emptyset \}$$

  is undecidable



Example 5

$$L = \{ \langle M \rangle : L(M) \text{ is regular} \}$$

undecidable

- Given M and w, construct a machine T with input z:

> On input $\langle M, w \rangle$ where M is a TM and w is a string
>
> Construct the following TM $T(z)$:
>
> Simulate M on input w
> If M halts on w
> T accepts z iff $z=0^i1^i$ for some $i \geq 0$

  - if M halts on w, $L(T)=\{0^i1^i\}$ which is not regular
  - if M does not halt on w, $L(T)=\emptyset$ which is regular
  - So if we can decide whether L(T) is regular then we can decide $HALT_{TM}$

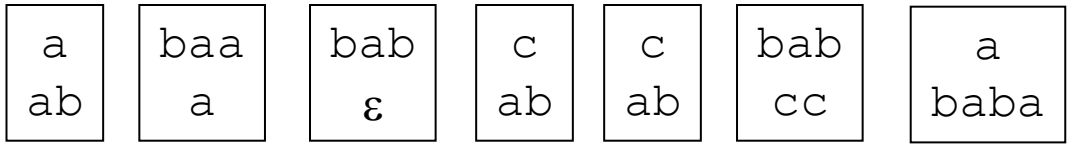


The Post Correspondence Problem

- Input: a set of tiles like this



- Given an unlimited supply of such tiles, can you find a match of top and bottom?





Undecidability of PCP

$$PCP = \{D: D \text{ is a collection of tiles that contains a top-bottom match}\}$$

- Theorem

> The language PCP is undecidable

- Proof: We will show that

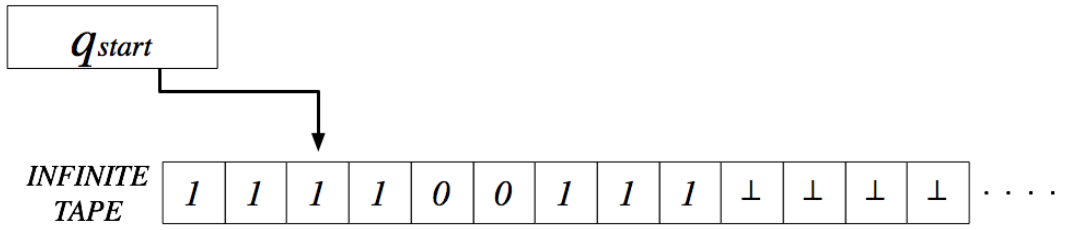
> If PCP can be decided, so can A_{TM}



Reductions via computation histories

- Computation history: sequence of configurations $C_1, C_2, \ldots$ (in the computing procedure of a TM)
  - So C_{i+1} is the successor of C_i
  - e.g. $C_i = u\ a\ q_i\ b\ v$, *and* $C_{i+1} = u\ q_j\ a\ c\ v$ if $\delta(q_i,b) = (q_j,c,L)$

A ***configuration*** is a list of the tape contents, head position, and state
e.g. configuration below is 11 q_{start} 1100111





Undecidability of PCP

$\langle M \rangle$, w $\longrightarrow$ T (collection of tiles)

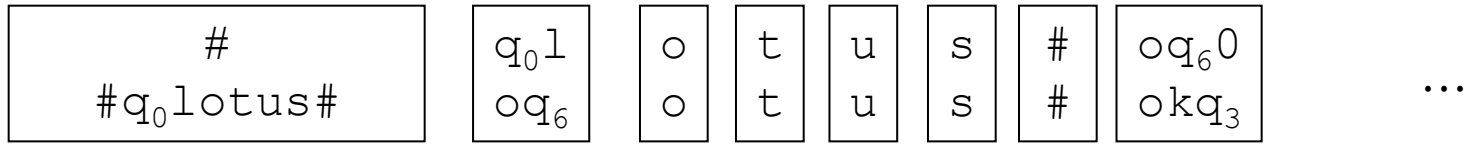
If M accepts w, then T can be matched

If M rej/loops on w, then T cannot be matched

- Idea: Matches represent accepting histories

$\#q_0lotus\#oq_6otus\#okq_3t...\#q_a\square\square\square\square$

$\#q_0lotus\#oq_6otus\#okq_3r...\#q_a\square\square\square\square$





Undecidability of PCP

$\langle M \rangle$, w ⟶ T (collection of tiles)

Put $\left[\frac{\#}{\#q_0 w\#}\right]$ into the piles if w is the input

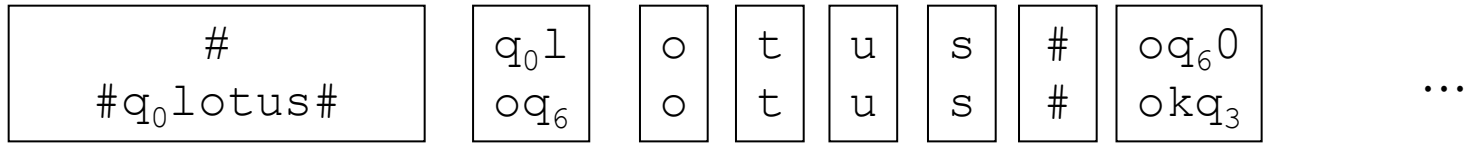
w, then T can be matched

s on w, then T cannot be matched

- Idea: Matches represent accepting histories

$\#q_0 lotus\#oq_6 otus\#okq_3 t...\#q_a \square\square\square\square$

$\#q_0 lotus\#oq_6 otus\#okq_3 r...\#q_a \square\square\square\square$





Undecidability of PCP

$\langle M \rangle$, w ⟶ T (collection of tiles)

Put $\left[\frac{\#}{\#q_0 w\#}\right]$ into the piles if w is the input

w, th

s on

$\left[\frac{qa}{br}\right]$ if **δ**(q,a)=(r,b,R)

$\left[\frac{cqa}{rcb}\right]$ if **δ**(q,a)=(r,b,L) for all c

- Idea: Matches represent accepting histories

$\left[\frac{a}{a}\right]$ for all symbol a

#$q_0$lotus#o$q_6$otus#okq_3t...#q

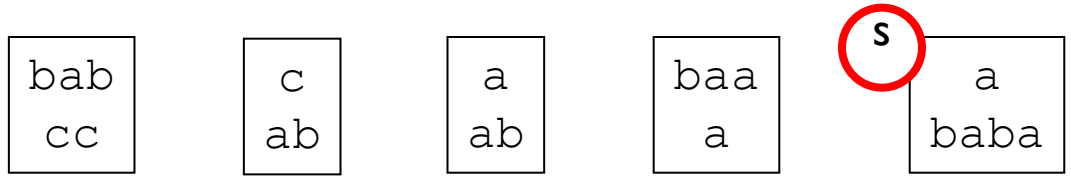
#$q_0$lotus#o$q_6$otus#okq_3r...#q_a□□□□

#	q_0l	o	t	u	s	#	o	$q_6$0	...
#$q_0$lotus#	oq_6	o	t	u	s	#	o	kq_3	



Some technicalities

- We will assume that
  - Before accepting, TM M erases its tape
  - One of the PCP tiles is marked as a starting tile



- These assumptions can be made without loss of generality (we will see why later)



Some technicalities

- We will assume that
  - Before accepting, TM M erases its tape
  - So the accept configuration will be $\#q_a\square\square\square\square\#$



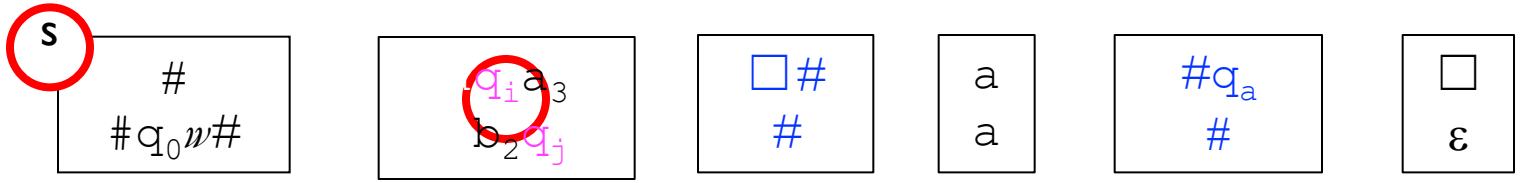
Undecidability of PCP

$\langle M \rangle$, w $\longrightarrow$ T (collection of tiles)

If M accepts w, then T can be matched

If M rej/loops on w, then T cannot be matched

- To decide A_{TM}, we construct these tiles for PCP



for each valid window of this form

for all a in $\Gamma \cup \{\#\}$

"final" tiles



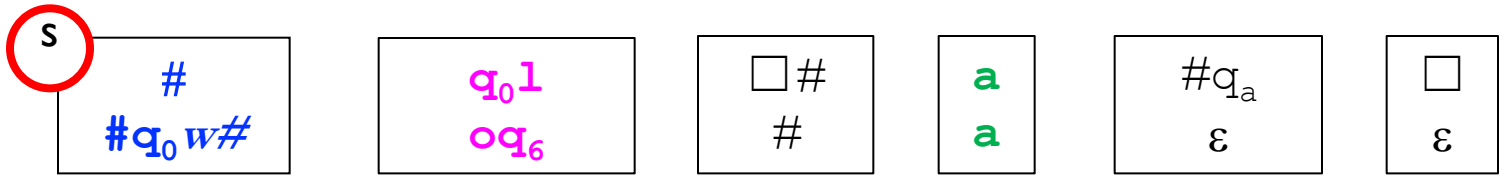
Undecidability of PCP

$\#q_0 lotus \# oq_6 otus \# ... \# oq_1 \square\square\square \# q_a \square\square\square\square$

accepting computation history

$\#q_0 lotus\#$

$\#q_0 lotus \# oq_6 otus\#$





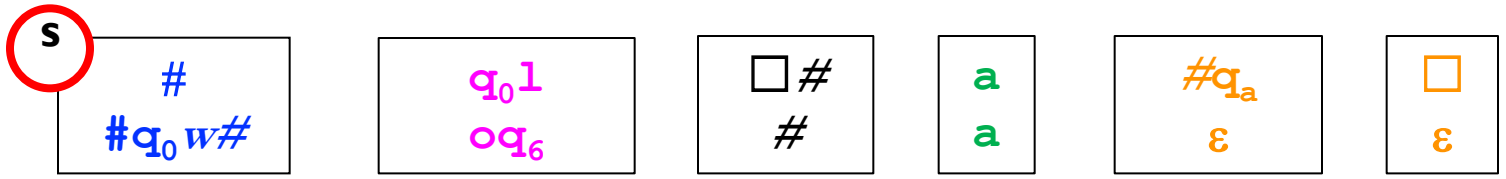
Undecidability of PCP

$\#q_0 lotus \# oq_6 otus \# ... \# oq_1 \square\square\square \# q_a \square\square\square\square$

accepting computation history

$\#q_0 lotus \# oq_6 otus \# ... \# oq_1 \square\square\square \# q_a \square\square\square\square$

$\#q_0 lotus \# oq_6 otus \# ... \# oq_1 \square\square\square \# q_a \square\square\square\square$

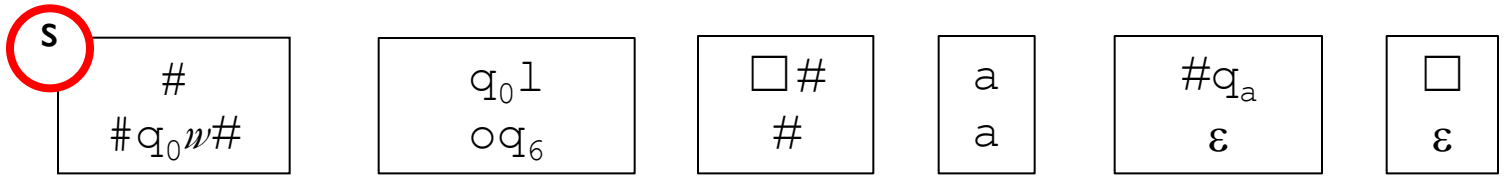




Undecidability of PCP

- If M rejects on input w, then q_r appears on bottom at some point, but it cannot be matched on top

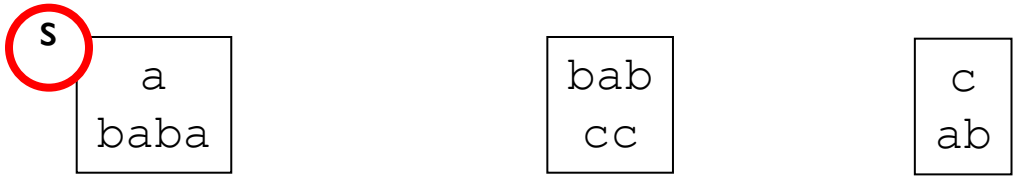
- If M loops on w, then matching keeps going forever





A technicality

- Remember we assumed that one tile marked as starting tile

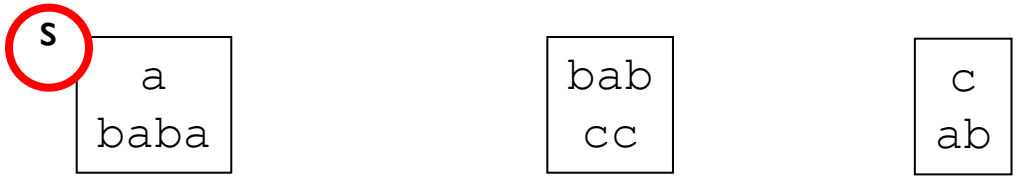


- How can we remove this assumption?

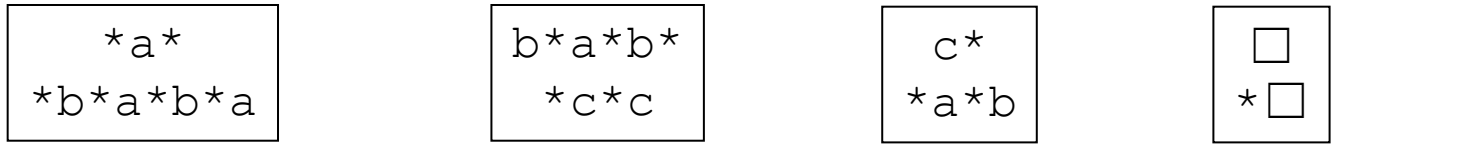


A technicality

- Remember we assumed that one tile marked as starting tile



- We can remove this assumption by changing tiles a little bit



"starting tile"
begins with *

"middle tiles"

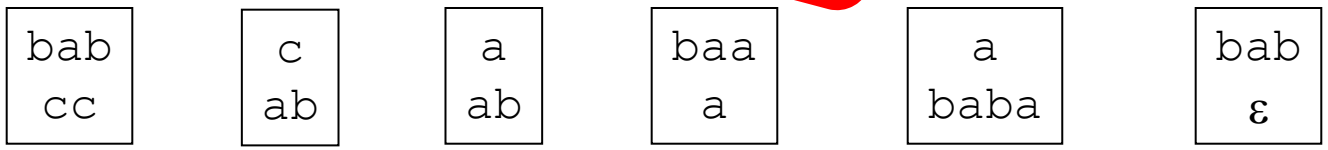
"ending tile"
matches last *



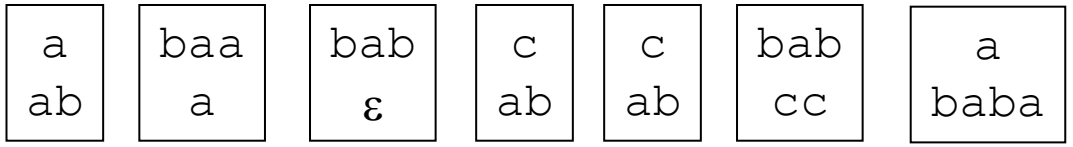
The Post Correspondence Problem

- Input: a set of tiles like this

undecidable



- Given an unlimited supply of such tiles, can you find a match of top and bottom?





Hilbert's 10th problem:

- Find an algorithm to determine whether a given polynomial Diophantine equation with integer coefficients has an integer solution
- Two or more variables

例：

$$x^3+y^3+z^3 = 33$$
$$x^3+y^3+z^3 = 42$$

存在：

$$8866128975287528^3 + (-8778405442862239)^3 + (-2736111468807040)^3 = 33$$
$$(-80538738812075974)^3 + 80435758145817515^3 + 12602123297335631^3 = 42$$

（2019年发现）



Diophantine set

- Diophantine equation: polynomial equation with integer coefficients
  - How to prove that solving Diophantine equation is not decidable?

- Diophantine set: for a Diophantine equation P with parameter *a*, the set of natural numbers for *a* making P have a natural number solution

$$a \in K \iff \left(\exists x \in \mathbb{N}^k\right)(P(a,x) = 0)$$

- e.g. {0,1,4,9,16,25,…} is a Diophantine set with equation $a - x^2 = 0$



MRDP theorem

- Diophantine set: for a Diophantine equation P with parameter *a*, the set of natural numbers for *a* making P have a natural number solution

$$a \in K \iff \left(\exists x \in \mathbb{N}^k\right)(P(a,x) = 0)$$

- *Matiyasevich–Robinson–Davis–Putnam (MRDP) theorem [1970]:*

  Every recursively enumerable set is Diophantine, and vice versa

  - Since there are recursively enumerable sets which are not recursive, so there are Diophantine sets which are not recursive



MRDP theorem

- *Matiyasevich–Robinson–Davis–Putnam (MRDP) theorem:*

  Every recursively enumerable set is Diophantine, and vice versa

The Diophantine equation for composite numbers:

$$a = (x_1 + 2)(x_2 + 2)$$

The Diophantine equations for prime numbers:

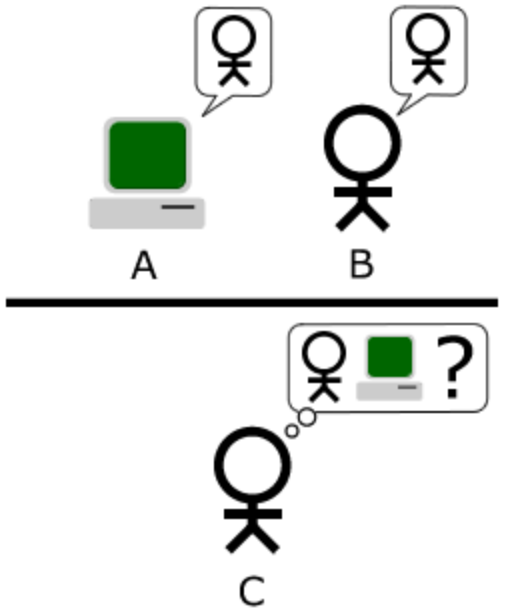
$$wz + h + j - q = 0$$
$$(gk + 2g + k + 1)(h + j) + h - z = 0$$
$$16(k+1)^3(k+2)(n+1)^2 + 1 - f^2 = 0$$
$$2n + p + q + z - e = 0$$
$$e^3(e+2)(a+1)^2 + 1 - o^2 = 0$$
$$(a^2 - 1)y^2 + 1 - x^2 = 0$$
$$16r^2y^4(a^2 - 1) + 1 - u^2 = 0$$
$$n + l + v - y = 0$$
$$(a^2 - 1)l^2 + 1 - m^2 = 0$$
$$ai + k + 1 - l - i = 0$$
$$\{[a + u^2(u^2 - a)]^2 - 1\}(n + 4dy)^2 + 1 - (x + cu)^2 = 0$$
$$p + l(a - n - 1) + b(2an + 2a - n^2 - 2n - 2) - m = 0$$
$$q + y(a - p - 1) + s(2ap + 2a - p^2 - 2p - 2) - x = 0$$
$$z + pl(a - p) + t(2ap - p^2 - 1) - pm = 0$$

k+2 is prime iff the 14 Diophantine equations in 26 variables have a solution in positive integers



Turing test

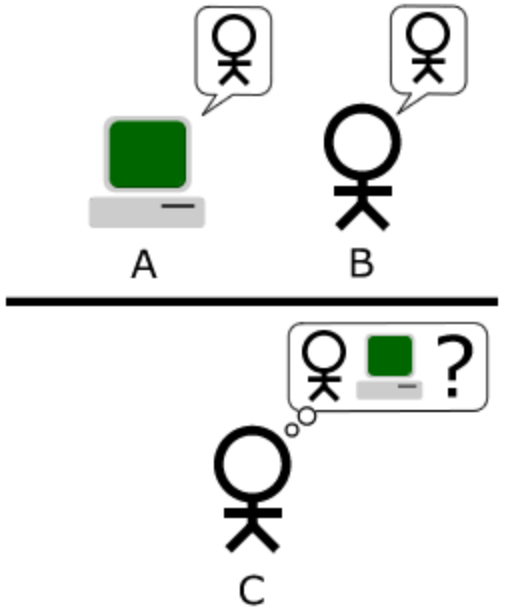
- The **Turing test** is a test of a machine's ability to exhibit intelligent behavior equivalent to, or indistinguishable from, that of a human





Turing test

- The test was introduced by Alan Turing in his 1950 paper "Computing Machinery and Intelligence", which opens with the words: "I propose to consider the question, 'Can machines think?'"



But: is it fair for computers? 😃



Turing test

- The test was introduced by Alan Turing in his 1950 paper "Computing Machinery and Intelligence", which opens with the words: "I propose to consider the question, 'Can machines think?'"

- More philosophically, can computer have "consciousness" (意识)
  - Strong AI: "The appropriately programmed computer with the right inputs and outputs would thereby have a mind in exactly the same sense human beings have minds"



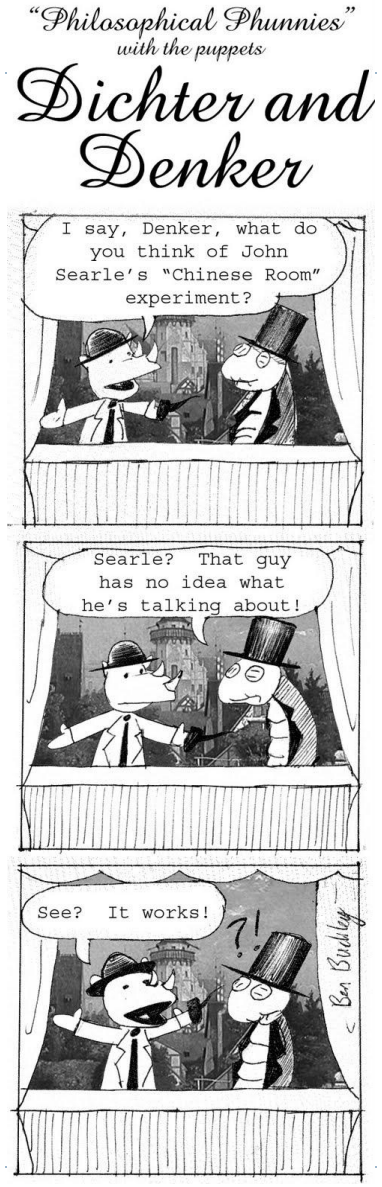
Chinese room

- A human don't know Chinese simulates the Turing machine which passes the Turing test in Chinese
  - Proposed by John Searle
  - Challenge the Strong AI: consciousness is algorithmic



Chinese room

- The computer doesn't seem to have consciousness even if it passes Turing test.



Turing test

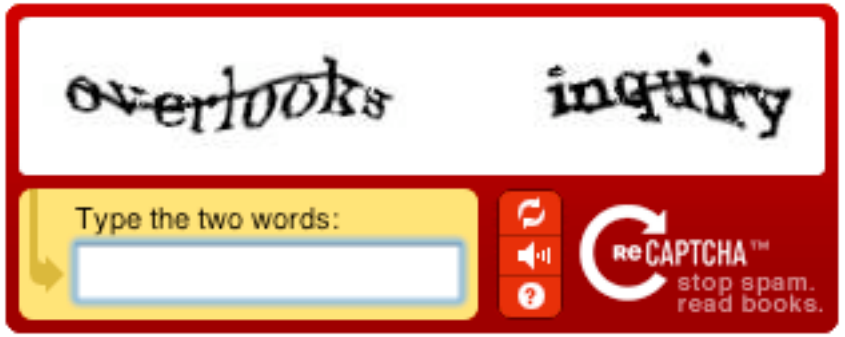
- The **Turing test** is a test of a machine's ability to exhibit intelligent behavior equivalent to, or indistinguishable from, that of a human

- Applications?



CAPTCHA

- “Completely Automated Public Turing Test To Tell Computers and Humans Apart”



  - A CAPTCHA is a program that protects websites against spammers by generating and grading tests that humans can pass but current computer programs cannot. *For example, humans can read distorted text as the one shown above, but current computer programs can't*

